Assignment Time Series

Zahra and Sanly

2026-06-21

Analyzing the Predictive Performance of Time Series Models on Brent Oil Prices

Crude oil, particularly Brent Crude Oil, plays a crucial role in the global economy as a primary energy source and the commodity with the highest trade volume. Fluctuations in crude oil prices are often used as an indicator of global economic conditions, which can impact many industrial sectors. Therefore, oil price prediction is crucial to minimize risk and support more informed decision-making. This study aims to develop a Brent Crude Oil price prediction model using several time series forecasting algorithms and compare their performance to determine the best model. Historical Brent Crude Oil closing price data from October 9, 2023, to April 30, 2026, was used in this study.

1. Import Library and Data

```
library(readxl)
library(dplyr)
library(ggplot2)
library(lubridate)
library(tseries)
library(car)
library(forecast)
library(zoo)
library(lmtest)
library(nortest)
library(tidyr)

MinyakBrent <- read_excel("Data Historis Minyak Brent Berjangka.xlsx")
head(MinyakBrent)
```

```
## # A tibble: 6 × 7
##   Tanggal             Terakhir Pembukaan Tertinggi Terendah Vol.
`Perubahan%`
##   <dttm>                 <dbl>     <dbl>     <dbl>    <dbl> <chr>
<dbl>
## 1 2026-04-30 00:00:00    109.      111.       115.    107.  469,41K      -
0.0114
## 2 2026-04-29 00:00:00    110.      104.       114.    103.  489,63K
0.0579
## 3 2026-04-28 00:00:00    104.      102.       106.    102.  428,95K
0.0266
## 4 2026-04-27 00:00:00    102.      101.       103.     99.6 334,94K
```

```
0.0258
## 5 2026-04-24 00:00:00     99.1     100.       101.     97.6 330,16K      -
0.0022
## 6 2026-04-23 00:00:00     99.4      95.9      101.     95.9 346,75K
0.033
```

The dataset used is historical Brent oil futures price data, consisting of several variables: trading date (Date), closing price (Last), opening price (Open), highest price (Highest), lowest price (Lowest), trading volume (Vol.), and daily price percentage change (Change%). The Date variable is of the time type (POSIXct), while the Price and Percentage Change variables are of numeric type. However, the date order in the data above is still from newest to oldest, so we need to arrange the order in ascending order.

2. Data Preprocessing

2.1 Sort data by date

```
MinyakBrent <- MinyakBrent[order(MinyakBrent$Tanggal),]
head(MinyakBrent)
```

```
## # A tibble: 6 × 7
##   Tanggal             Terakhir Pembukaan Tertinggi Terendah Vol.
`Perubahan%`
##   <dttm>                 <dbl>     <dbl>     <dbl>    <dbl> <chr>
<dbl>
## 1 2023-10-09 00:00:00     88.2      86.4      89       86   412,76K
0.0422
## 2 2023-10-10 00:00:00     87.6      88.2      88.5     86.9 352,11K      -
0.0057
## 3 2023-10-11 00:00:00     85.8      87.7      88.3     85.2 402,31K      -
0.0209
## 4 2023-10-12 00:00:00     86        85.5      87.6     85.2 370,44K
0.0021
## 5 2023-10-13 00:00:00     90.9      86.4      91       86.3 424,71K
0.0569
## 6 2023-10-16 00:00:00     89.6      91.0      91.4     89.5 283,25K      -
0.0136
```

2.2 Selecting the variables used

```
MinyakBrent <- MinyakBrent %>%
  select(Tanggal, Terakhir) %>%
  mutate(Tanggal = as.Date(Tanggal))
```

In this study, we will focus on examining the variable 'Terakhir' because we want to see how the last price of Brent oil is each day.

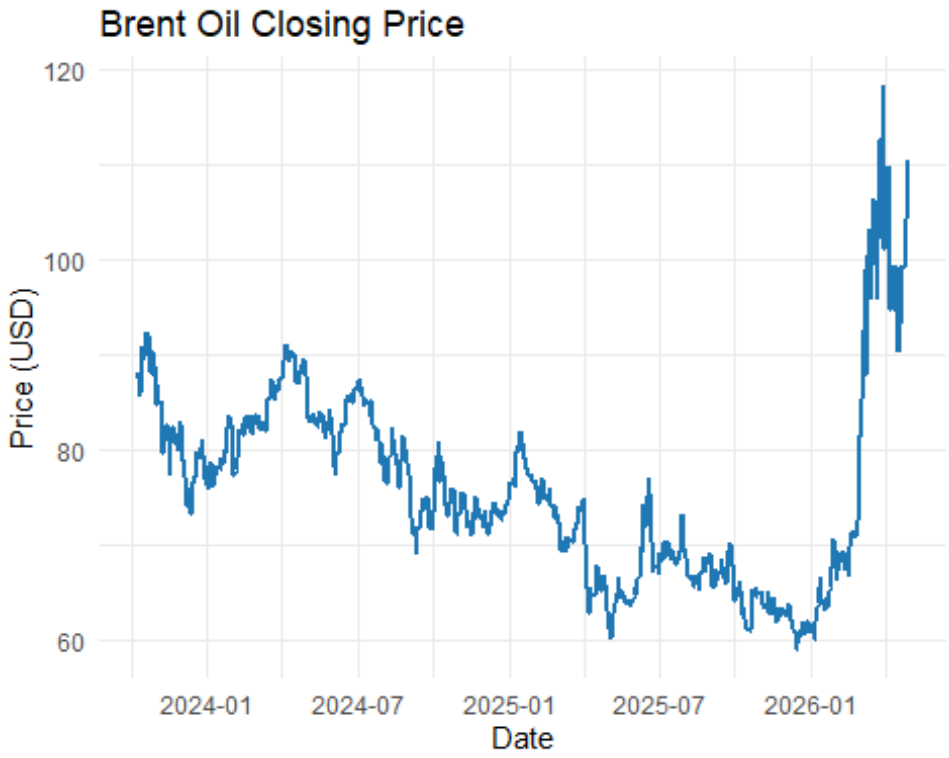
2.3 Checking missing values

```
colSums(is.na(MinyakBrent))
```

```
##  Tanggal Terakhir
##        0        0
```

Based on the results of the examination, no missing values were found in all variables so that the data can be used directly for the next analysis stage.

```
ggplot(MinyakBrent,
       aes(x = Tanggal,
           y = Terakhir)) +
  geom_line(
    color = "#1f77b4",
    linewidth = 1
  ) +
  labs(
    title = "Brent Oil Closing Price",
    x = "Date",
    y = "Price (USD)"
  ) +
  theme_minimal()
```



Based on the Brent oil price chart for the 2023–2026 period, it appears that oil prices experienced quite dynamic fluctuations throughout the observation period. In 2023, Brent prices hovered in the range of USD 75–90 per barrel, remaining relatively stable despite several short-term spikes and dips. Entering 2024, prices tended to gradually weaken, moving within a lower range than the previous year, around USD 70–80 per barrel.

In 2025, the downward trend continued, with prices several times touching levels below USD 70 per barrel. This period indicated relatively weaker market conditions compared to the previous two years. However, towards the end of 2025, a price recovery began to emerge, marked by a gradual increase.

The most significant change occurred in 2026, when Brent oil prices experienced a sharp surge from around USD 70 per barrel to over USD 110 per barrel. After reaching that high point, the price remained at a relatively high level but showed greater movement compared to the previous period.

2.4 Checking the Data Structure

```
str(MinyakBrent)
```

```
## tibble [660 × 2] (S3: tbl_df/tbl/data.frame)
##  $ Tanggal : Date[1:660], format: "2023-10-09" "2023-10-10" ...
##  $ Terakhir: num [1:660] 88.2 87.7 85.8 86 90.9 ...
```

The date variable is of type POSIXct while the price variable is of type numeric so it is suitable for time series analysis.

```
summary(MinyakBrent)
```

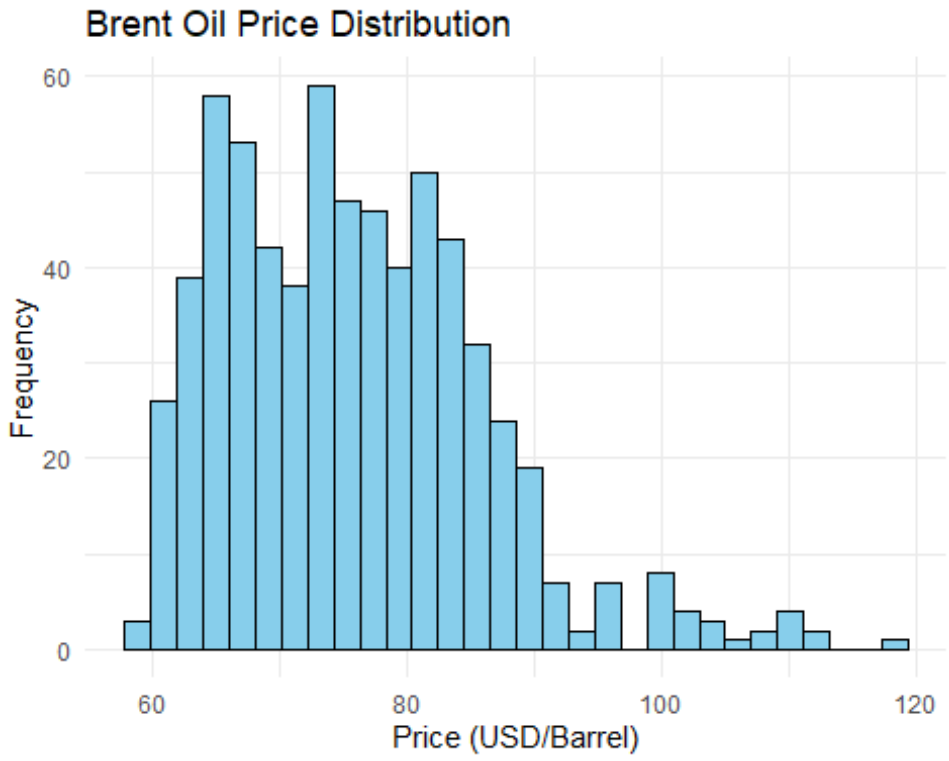
```
##     Tanggal              Terakhir
##  Min.   :2023-10-09   Min.   : 58.92
##  1st Qu.:2024-05-29   1st Qu.: 67.63
##  Median :2025-01-18   Median : 74.65
##  Mean   :2025-01-17   Mean   : 75.91
##  3rd Qu.:2025-09-08   3rd Qu.: 82.37
##  Max.   :2026-04-30   Max.   :118.35
```

The average Brent oil price during the observation period was US$75.91 per barrel, with a minimum price of US$58.92 per barrel and a maximum price of US$118.35 per barrel. This indicates significant price fluctuations during the observation period.

3. Exploratory Data Analysis (EDA)

3.1 Histogram Harga Minyak Brent

```
ggplot(MinyakBrent, aes(x = Terakhir)) +
  geom_histogram(
    bins = 30,
    fill = "skyblue",
    color = "black"
  ) +
  labs(
    title = "Brent Oil Price Distribution",
    x = "Price (USD/Barrel)",
    y = "Frequency"
  ) +
  theme_minimal()
```

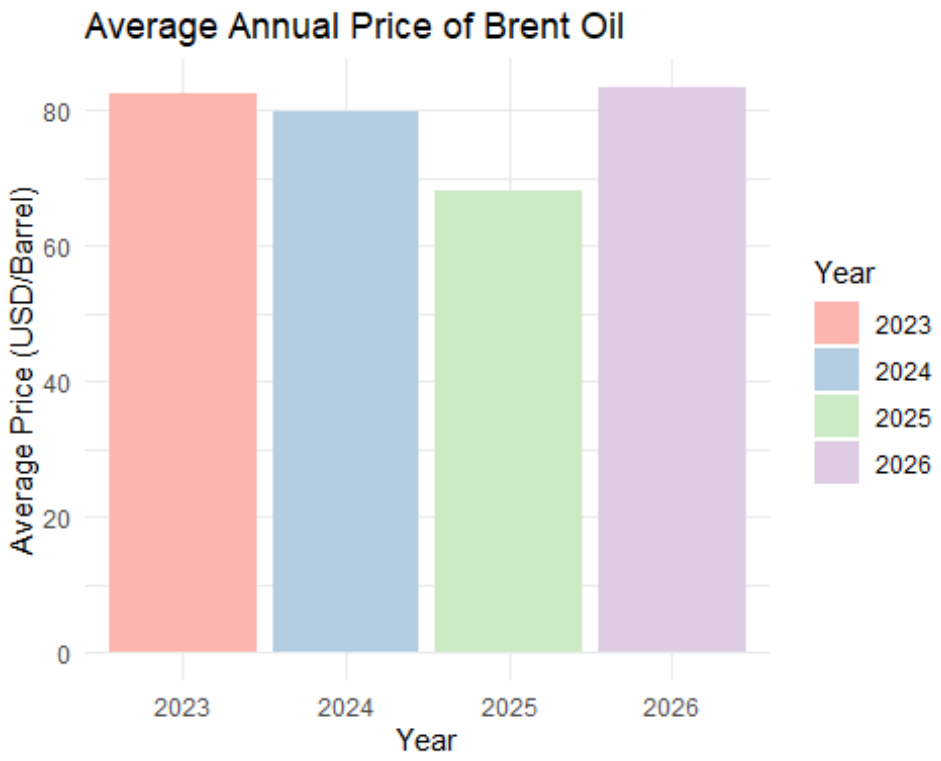


Based on the histogram shows that Brent oil prices are primarily concentrated between USD 60 and USD 85 per barrel. The distribution is positively skewed, with a longer right tail indicating the presence of several high-price observations above USD 100 per barrel. This suggests that extreme price increases occurred occasionally during the observation period, while most prices remained within a moderate range.

3.2 Barplot Harga Minyak Brent per Tahun

```
harga_tahunan <- MinyakBrent %>%
  mutate(Tahun = year(Tanggal)) %>%
  group_by(Tahun) %>%
  summarise(
    Rata_Rata = mean(Terakhir, na.rm = TRUE)
  )

ggplot(harga_tahunan,
       aes(x = factor(Tahun),
           y = Rata_Rata,
           fill = factor(Tahun))) +
  geom_col() +
  scale_fill_brewer(palette = "Pastel1") +
  labs(
    title = "Average Annual Price of Brent Oil",
    x = "Year",
    y = "Average Price (USD/Barrel)",
    fill = "Year"
```

```
  ) +
  theme_minimal()
```

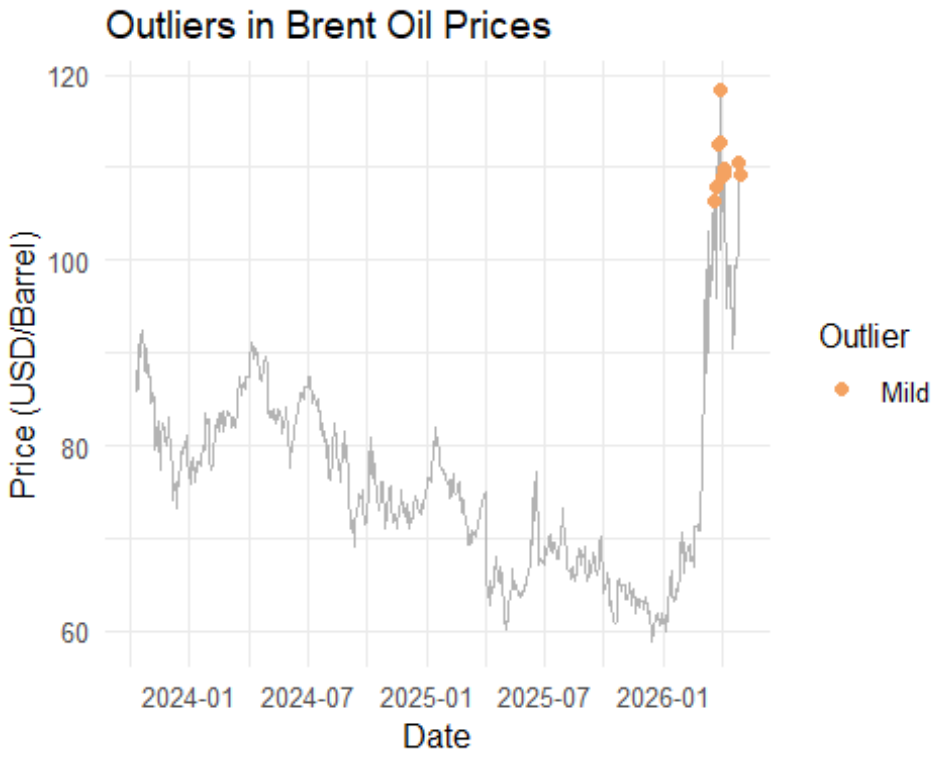


The annual average Brent oil prices exhibit a fluctuating pattern rather than a consistent upward or downward trend. The average price declined between 2023 and 2025, reaching its lowest level in 2025. Subsequently, a notable recovery occurred in 2026, resulting in the highest annual average price during the observation period. This pattern suggests that Brent oil prices are influenced by changing market conditions and external economic factors, leading to variations in price levels across different years.

3.3 Outlier Plot

```
Q1 <- quantile(MinyakBrent$Terakhir, 0.25)
Q3 <- quantile(MinyakBrent$Terakhir, 0.75)

IQR_value <- IQR(MinyakBrent$Terakhir)

lower_mild <- Q1 - 1.5 * IQR_value
upper_mild <- Q3 + 1.5 * IQR_value

lower_extreme <- Q1 - 3 * IQR_value
upper_extreme <- Q3 + 3 * IQR_value

MinyakBrent$Outlier <- "Normal"

MinyakBrent$Outlier[
  MinyakBrent$Terakhir < lower_mild |
```

```
  MinyakBrent$Terakhir > upper_mild
] <- "Mild"

MinyakBrent$Outlier[
  MinyakBrent$Terakhir < lower_extreme |
  MinyakBrent$Terakhir > upper_extreme
] <- "Extreme"

ggplot(MinyakBrent, aes(x = Tanggal, y = Terakhir)) +

  geom_line(
    color = "grey70",
    linewidth = 0.3
  ) +

  geom_point(
    data = subset(MinyakBrent, Outlier != "Normal"),
    aes(color = Outlier),
    size = 2
  ) +

  scale_color_manual(
    values = c(
      "Mild" = "#f4a261",
      "Extreme" = "#d73027"
    )
  ) +

  labs(
    title = "Outliers in Brent Oil Prices",
    x = "Date",
    y = "Price (USD/Barrel)",
    color = "Outlier"
  ) +

  theme_minimal(base_size = 12)
```



4. Time series conversion

In order to be analyzed, the Brent Oil variable was converted into time series data using the ts() function.

The time series object is formed using the closing price variable because this variable is the most commonly used in the analysis and forecasting of financial asset prices.

```
ts_data <- ts(MinyakBrent$Terakhir)
ts_data
```

```
## Time Series:
## Start = 1
## End = 660
## Frequency = 1
##   [1]  88.15  87.65  85.82  86.00  90.89  89.65  89.90  91.50  92.38
92.16
##  [11]  89.83  88.07  90.13  87.93  90.48  87.45  87.41  84.63  86.85
84.89
##  [21]  85.18  81.61  79.54  80.01  81.43  82.52  82.47  81.18  77.42
80.61
##  [31]  82.32  82.45  81.96  81.42  80.58  79.98  81.68  83.10  82.83
78.88
##  [41]  78.03  77.20  74.30  74.05  75.84  76.03  73.24  74.26  76.61
76.55
##  [51]  77.95  79.23  79.70  79.39  79.07  81.07  79.65  78.39  77.04
75.89
```

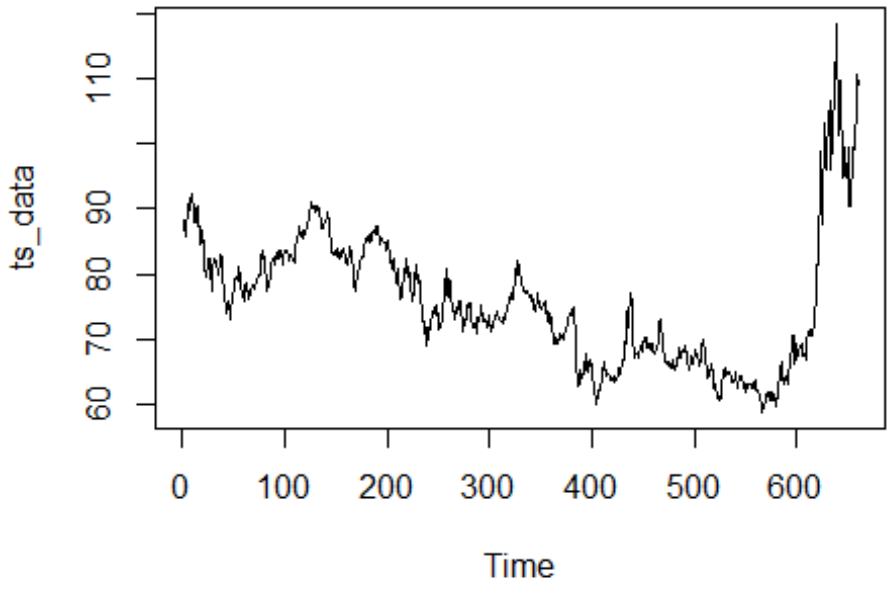
```
##  [61]  78.25  77.59  78.76  76.12  77.59  76.80  77.41  78.29  78.15
78.29
##  [71]  77.88  79.10  78.56  80.06  79.55  80.04  82.43  83.55  82.40
82.87
##  [81]  81.71  78.70  77.33  77.99  78.59  79.21  81.63  82.19  82.00
82.77
##  [91]  81.60  82.86  83.47  83.56  82.34  83.03  83.67  81.62  82.53
83.65
## [101]  83.68  83.62  83.55  82.80  82.04  82.96  82.96  82.08  82.21
81.92
## [111]  84.03  85.42  85.34  86.89  87.38  85.95  85.78  85.43  86.75
86.25
## [121]  86.09  87.48  87.42  88.92  89.35  90.65  91.17  90.38  89.42
90.48
## [131]  89.74  90.45  90.10  90.02  87.29  87.11  87.29  87.00  88.42
88.02
## [141]  89.01  89.50  88.40  87.86  83.44  83.67  82.96  83.33  83.16
83.58
## [151]  83.88  82.79  83.36  82.38  82.75  83.27  83.98  83.71  82.88
81.90
## [161]  81.36  82.12  83.10  84.22  83.60  81.86  81.62  78.36  77.52
78.41
## [171]  79.87  79.62  81.63  81.92  82.60  82.75  82.62  84.25  85.33
85.07
## [181]  85.71  85.24  86.01  85.01  85.25  86.39  86.41  86.60  86.24
87.34
## [191]  87.43  86.54  85.75  84.66  85.08  85.40  85.03  84.85  83.73
85.08
## [201]  85.11  82.63  82.40  81.01  81.71  82.37  81.13  79.78  78.63
80.72
## [211]  79.52  76.81  76.30  76.48  78.33  79.16  79.66  82.30  80.69
79.76
## [221]  81.04  79.68  77.66  77.20  76.05  77.22  79.02  81.43  79.55
78.65
## [231]  79.94  78.80  77.52  73.75  72.70  72.69  71.06  71.84  69.19
70.61
## [241]  71.97  71.61  72.75  73.70  73.65  74.88  74.49  73.90  75.17
73.46
## [251]  71.60  71.98  71.77  73.56  73.90  77.62  78.05  80.93  77.18
76.58
## [261]  79.40  79.04  77.46  74.25  74.22  74.45  73.06  74.29  76.04
74.96
## [271]  74.38  76.05  71.42  71.12  72.55  73.16  73.10  75.08  75.53
74.92
## [281]  75.63  73.87  71.83  71.89  72.28  72.56  71.04  73.30  73.31
72.81
## [291]  74.23  75.17  73.01  72.81  72.83  73.28  72.94  71.83  73.62
72.31
## [301]  72.09  71.12  72.14  72.19  73.52  73.41  74.49  73.91  73.19
73.39
```

```
## [311]  72.88  72.94  72.63  73.58  73.26  74.17  74.39  74.64  75.93
76.51
## [321]  76.30  77.05  76.16  76.92  79.76  81.01  79.92  82.03  81.29
80.79
## [331]  79.10  78.40  78.15  77.56  77.55  77.08  77.49  76.58  76.87
76.76
## [341]  75.96  76.20  74.61  74.29  74.66  75.87  77.00  75.18  75.02
74.74
## [351]  74.93  75.43  75.67  76.05  74.05  74.31  73.02  72.53  74.04
73.18
## [361]  71.62  71.04  69.30  69.46  70.36  69.28  69.56  70.95  69.88
70.58
## [371]  70.60  70.12  70.32  71.47  71.61  72.37  73.02  73.79  74.03
73.63
## [381]  74.74  74.49  74.95  70.14  65.58  64.21  62.82  65.48  63.33
64.76
## [391]  64.88  64.67  65.85  67.96  65.30  66.48  65.18  66.55  66.87
65.86
## [401]  64.25  63.12  62.13  61.29  60.23  62.15  61.12  62.84  63.91
64.96
## [411]  66.63  66.09  64.53  65.41  64.80  64.76  64.38  63.92  64.21
64.12
## [421]  63.57  64.32  64.15  63.90  64.63  65.63  64.86  65.34  66.47
67.04
## [431]  66.87  69.77  69.36  74.23  71.99  74.90  75.14  77.08  75.48
70.52
## [441]  67.14  67.68  67.73  67.77  67.61  67.11  69.11  68.80  68.30
69.58
## [451]  70.15  70.19  68.64  70.36  69.21  68.71  68.52  69.52  69.28
68.38
## [461]  67.77  67.83  68.36  68.44  70.04  72.51  73.24  72.53  69.67
68.76
## [471]  67.64  66.89  66.43  66.59  66.63  66.12  65.63  66.84  65.85
66.01
## [481]  65.30  66.31  67.12  67.22  68.80  67.22  68.05  68.62  68.12
68.15
## [491]  69.14  67.60  66.99  65.50  66.02  66.39  67.49  66.37  66.99
67.44
## [501]  68.47  67.95  67.44  66.68  65.97  66.97  69.31  69.42  70.13
67.97
## [511]  67.02  65.35  64.11  64.53  65.47  65.45  66.25  65.22  62.73
63.32
## [521]  62.39  61.91  61.06  61.29  60.88  61.03  62.23  65.29  65.20
65.62
## [531]  64.40  64.92  65.00  65.07  64.89  64.44  63.52  63.38  63.63
64.06
## [541]  65.16  62.71  63.01  64.39  63.76  64.43  63.00  62.80  61.94
63.37
## [551]  62.48  63.13  63.34  63.20  63.17  62.45  62.67  63.26  63.75
62.49
```

```
## [561]  61.94  62.21  61.28  61.12  60.56  58.92  59.68  59.82  60.47
61.58
## [571]  61.87  61.80  60.64  61.94  61.92  60.85  60.75  61.76  60.70
59.96
## [581]  61.99  63.34  63.87  65.47  66.52  63.76  64.13  63.22  64.19
64.53
## [591]  63.34  65.07  65.59  67.57  68.40  70.71  70.69  66.30  67.33
69.46
## [601]  67.55  68.05  69.04  68.80  69.40  67.52  67.75  67.97  66.87
69.77
## [611]  71.27  71.30  71.49  70.77  70.85  70.75  72.48  77.74  81.40
81.40
## [621]  85.41  92.69  98.96  87.80  91.98 100.46 103.14  96.04  99.39
102.92
## [631] 103.78 106.41  95.92 100.23 102.22 108.01 112.57 112.78 118.35
101.16
## [641] 109.03 109.77 109.27  94.75  95.92  95.20  99.36  94.79  94.93
99.39
## [651]  90.38  90.43  93.24  96.18  99.35  99.13 101.69 104.40 110.44
109.18
```

Visualization Brent Oil Price

```
plot(ts_data)
```



Based on the Brent oil price chart for the 2023–2026 period, it appears that oil prices experienced quite dynamic fluctuations throughout the observation period. In 2023, Brent prices hovered in

the range of USD 75–90 per barrel, remaining relatively stable despite several short-term spikes and dips. Entering 2024, prices tended to gradually weaken, moving within a lower range than the previous year, around USD 70–80 per barrel.

In 2025, the downward trend continued, with prices several times touching levels below USD 70 per barrel. This period indicated relatively weaker market conditions compared to the previous two years. However, towards the end of 2025, a price recovery began to emerge, marked by a gradual increase.

The most significant change occurred in 2026, when Brent oil prices experienced a sharp surge from around USD 70 per barrel to over USD 110 per barrel. After reaching that high point, the price remained at a relatively high level but showed greater movement compared to the previous period.

5. Split Data (80% Training, 20% Testing)

```
n_total <- length(ts_data)
train_size <- round(n_total * 0.8)
test_size <- n_total - train_size

train_data <- ts(ts_data[1:train_size])
test_data <- ts(ts_data[(train_size + 1):n_total], start = train_size + 1)

cat("Jumlah Data Training:", length(train_data), "\n")
```

```
## Jumlah Data Training: 528
```

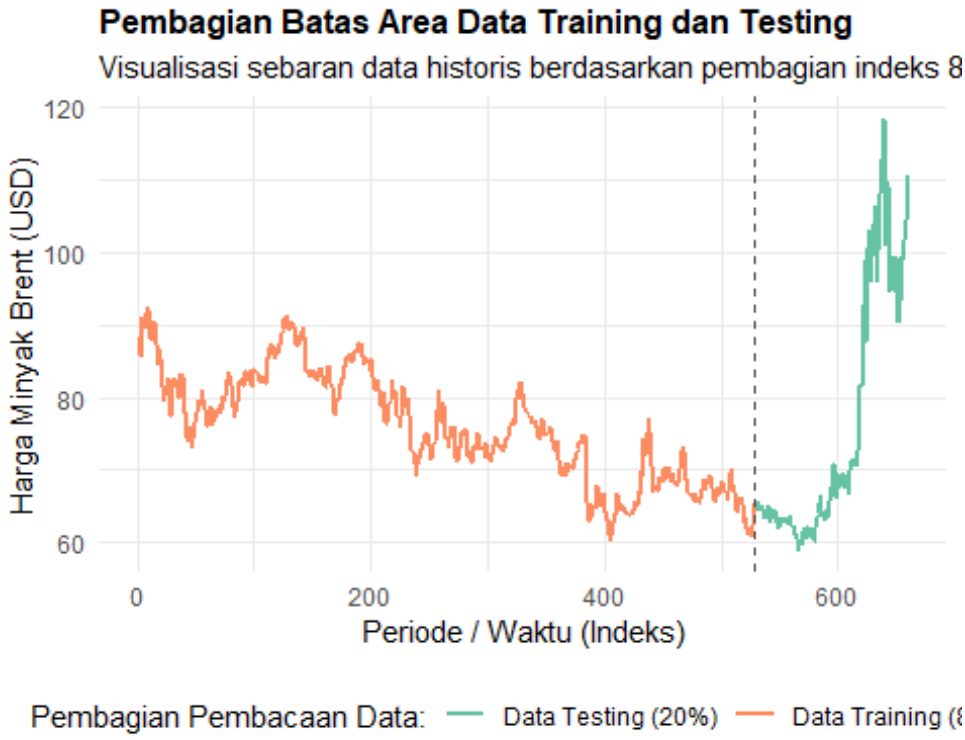
```
cat("Jumlah Data Testing :", length(test_data), "\n")
```

```
## Jumlah Data Testing : 132
```

Visualization splitting data

```
df_kronologis <- data.frame(
  Indeks = 1:length(ts_data),
  Harga  = as.numeric(ts_data),
  Kelompok = c(rep("Data Training (80%)", train_size), rep("Data Testing
(20%)", test_size))
)

ggplot(df_kronologis, aes(x = Indeks, y = Harga, color = Kelompok)) +
  geom_line(linewidth = 0.8) +
  geom_vline(xintercept = train_size, linetype = "dashed", color = "gray30",
linewidth = 0.7) +
  scale_color_brewer(palette = "Set2") +
  labs(
    title = "Pembagian Batas Area Data Training dan Testing",
    subtitle = "Visualisasi sebaran data historis berdasarkan pembagian
indeks 80:20",
    x = "Periode / Waktu (Indeks)",
```

```
    y = "Harga Minyak Brent (USD)",
    color = "Pembagian Pembacaan Data:"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 12),
    legend.position = "bottom"
  )
```



6. Stasionarity Analysis

6.1 Stationarity Test to Variance

H_0: Data is stationary to variance H_1: Data is not stationary to variance α = 5% = 0.05

```
summary(powerTransform(train_data))
```

```
## bcPower Transformation to Normality
##            Est Power Rounded Pwr Wald Lwr Bnd Wald Upr Bnd
## train_data    0.7227           1      -0.1488       1.5941
##
## Likelihood ratio test that transformation parameter is equal to 0
##  (log transformation)
##                             LRT df    pval
## LR test, lambda = (0) 2.646648  1 0.10377
##
## Likelihood ratio test that no transformation is needed
```

```
##                               LRT df    pval
## LR test, lambda = (1) 0.3886813  1 0.53299
```

See that lambda = 1. Reject H_0 if p-value is less than α. Because p-value (0.53299) is more than α (0.05), so we failed to reject the H_0. This means that the data is already stationary to variance, so we don't need to transform the data.

6.2 Stationarity Test to Mean

H_0 : Data is not stationary to the mean H_1 : Data is stationary to the mean α = 5% = 0.05

```
adf.test(train_data)

##
##  Augmented Dickey-Fuller Test
##
## data:  train_data
## Dickey-Fuller = -3.2225, Lag order = 8, p-value = 0.08405
## alternative hypothesis: stationary
```

Reject H_0 if p-value less than α. because p-value (0.08405) is more than α (0.05), we failed to reject H_0. This means The ADF test results on the Box-Cox transformed data indicate that the data is still not stationary about the mean. Therefore, first-order differencing is performed to eliminate trends and stabilize the mean. After differencing, the Augmented Dickey-Fuller (ADF) test is performed again to evaluate stationarity about the mean.

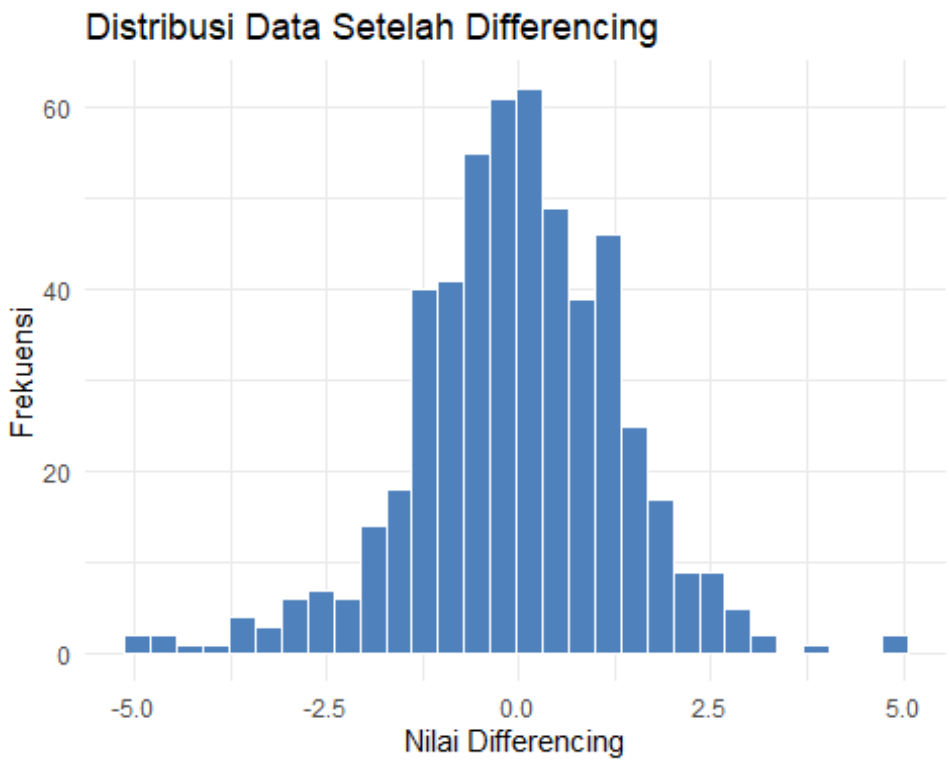
```
ts_diff <- diff(train_data)
adf.test(ts_diff)

## Warning in adf.test(ts_diff): p-value smaller than printed p-value

##
##  Augmented Dickey-Fuller Test
##
## data:  ts_diff
## Dickey-Fuller = -9.8386, Lag order = 8, p-value = 0.01
## alternative hypothesis: stationary
```

Reject H_0 if p-value is less than α. because the p-value (0.01) is less than α (0.05), we reject H_0. This means that the data is stationary to the mean.

This means that the data is ready to be used because it meets the stationarity assumption.

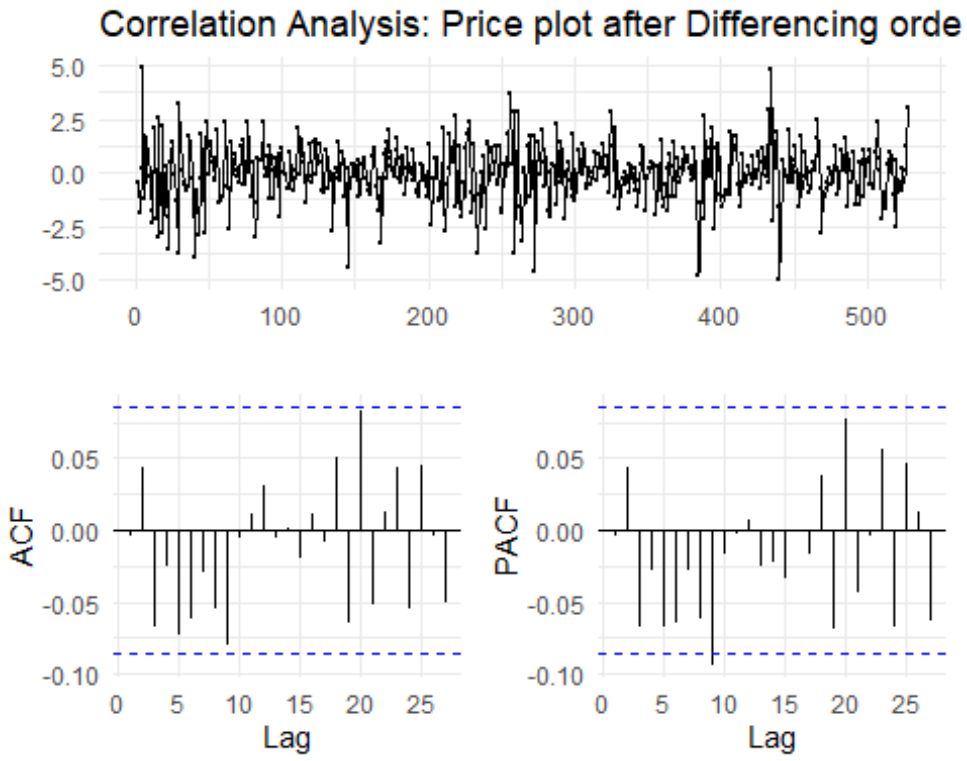
```
diff_df <- data.frame(
  Differencing = as.numeric(ts_diff)
)

ggplot(diff_df,
       aes(x = Differencing)) +
  geom_histogram(
    bins = 30,
```

```
    fill = "#4F81BD",
    color = "white"
  ) +
  labs(
    title = "Distribusi Data Setelah Differencing",
    x = "Nilai Differencing",
    y = "Frekuensi"
  ) +
  theme_minimal()
```



7. Identify Model

```
ggtsdisplay(ts_diff,
            main = "Correlation Analysis: Price plot after Differencing order
1, ACF, and PACF",
            theme = theme_minimal())
```



The graphs of the Box-Cox transformed and first-order differencing data show that fluctuations have centered around a relatively constant mean and no longer exhibit a clear trend. Furthermore, the data distribution tends to be more stable than before the transformation, although some spikes remain during certain periods.

The stationarity of the first-order differenced series (ΔY_t) is visually confirmed by the time series plot (top panel), which fluctuates around a constant mean of zero with no discernible trend. To identify the potential orders for the Autoregressive (p) and Moving Average (q) components of the ARIMA model, the Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) plots are analyzed.

The ACF plot is utilized to determine the maximum order of the Moving Average (q) process. As shown in the bottom-left panel, nearly all spikes at the initial lags (lags 1 to 8) remain strictly within the 95% confidence intervals (represented by the blue dashed lines). Although there is a minor significant spike at lag 9 and lag 20, the lack of significant, persistent correlation at the earliest lags suggests that the moving average component is negligible or close to zero.

The PACF plot is employed to identify the order of the Autoregressive (p) process. Similar to the ACF behavior, the PACF plot (bottom-right panel) reveals that the partial autocorrelations at the immediate past lags are statistically insignificant, failing to cross the threshold boundaries. A single spike slightly breaches the confidence interval at lag 9, which is typical for high-frequency financial time series and can often be attributed to random noise or minor seasonal market microstructures rather than a true structural AR process.

8. Time Series Forecasting (Traditional Method)

8.1 Naive Forecast

```
h_period <- n_total - train_size
model_naive <- naive(train_data, h = h_period)
print(model_naive)
```

```
##     Point Forecast    Lo 80    Hi 80    Lo 95    Hi 95
## 529          65.29 63.54846 67.03154 62.62655 67.95345
## 530          65.29 62.82710 67.75290 61.52331 69.05669
## 531          65.29 62.27357 68.30643 60.67677 69.90323
## 532          65.29 61.80693 68.77307 59.96310 70.61690
## 533          65.29 61.39581 69.18419 59.33435 71.24565
## 534          65.29 61.02413 69.55587 58.76591 71.81409
## 535          65.29 60.68233 69.89767 58.24318 72.33682
## 536          65.29 60.36419 70.21581 57.75663 72.82337
## 537          65.29 60.06539 70.51461 57.29965 73.28035
## 538          65.29 59.78278 70.79722 56.86743 73.71257
## 539          65.29 59.51398 71.06602 56.45634 74.12366
## 540          65.29 59.25714 71.32286 56.06354 74.51646
## 541          65.29 59.01080 71.56920 55.68680 74.89320
## 542          65.29 58.77377 71.80623 55.32429 75.25571
## 543          65.29 58.54506 72.03494 54.97451 75.60549
## 544          65.29 58.32386 72.25614 54.63620 75.94380
## 545          65.29 58.10946 72.47054 54.30832 76.27168
## 546          65.29 57.90129 72.67871 53.98994 76.59006
## 547          65.29 57.69882 72.88118 53.68029 76.89971
## 548          65.29 57.50162 73.07838 53.37869 77.20131
## 549          65.29 57.30928 73.27072 53.08454 77.49546
## 550          65.29 57.12147 73.45853 52.79732 77.78268
## 551          65.29 56.93789 73.64211 52.51655 78.06345
## 552          65.29 56.75825 73.82175 52.24182 78.33818
## 553          65.29 56.58232 73.99768 51.97275 78.60725
## 554          65.29 56.40987 74.17013 51.70902 78.87098
## 555          65.29 56.24071 74.33929 51.45031 79.12969
## 556          65.29 56.07466 74.50534 51.19635 79.38365
## 557          65.29 55.91154 74.66846 50.94689 79.63311
## 558          65.29 55.75122 74.82878 50.70169 79.87831
## 559          65.29 55.59354 74.98646 50.46054 80.11946
## 560          65.29 55.43839 75.14161 50.22326 80.35674
## 561          65.29 55.28564 75.29436 49.98965 80.59035
## 562          65.29 55.13519 75.44481 49.75956 80.82044
## 563          65.29 54.98694 75.59306 49.53282 81.04718
## 564          65.29 54.84079 75.73921 49.30930 81.27070
## 565          65.29 54.69665 75.88335 49.08887 81.49113
## 566          65.29 54.55445 76.02555 48.87140 81.70860
## 567          65.29 54.41411 76.16589 48.65677 81.92323
## 568          65.29 54.27556 76.30444 48.44487 82.13513
## 569          65.29 54.13873 76.44127 48.23560 82.34440
```

```
## 570          65.29 54.00356 76.57644 48.02888 82.55112
## 571          65.29 53.86999 76.71001 47.82460 82.75540
## 572          65.29 53.73796 76.84204 47.62268 82.95732
## 573          65.29 53.60742 76.97258 47.42304 83.15696
## 574          65.29 53.47833 77.10167 47.22561 83.35439
## 575          65.29 53.35063 77.22937 47.03031 83.54969
## 576          65.29 53.22429 77.35571 46.83708 83.74292
## 577          65.29 53.09925 77.48075 46.64586 83.93414
## 578          65.29 52.97548 77.60452 46.45657 84.12343
## 579          65.29 52.85295 77.72705 46.26917 84.31083
## 580          65.29 52.73161 77.84839 46.08359 84.49641
## 581          65.29 52.61143 77.96857 45.89980 84.68020
## 582          65.29 52.49238 78.08762 45.71773 84.86227
## 583          65.29 52.37442 78.20558 45.53733 85.04267
## 584          65.29 52.25754 78.32246 45.35857 85.22143
## 585          65.29 52.14169 78.43831 45.18140 85.39860
## 586          65.29 52.02686 78.55314 45.00578 85.57422
## 587          65.29 51.91301 78.66699 44.83166 85.74834
## 588          65.29 51.80012 78.77988 44.65901 85.92099
## 589          65.29 51.68817 78.89183 44.48780 86.09220
## 590          65.29 51.57713 79.00287 44.31798 86.26202
## 591          65.29 51.46699 79.11301 44.14953 86.43047
## 592          65.29 51.35771 79.22229 43.98241 86.59759
## 593          65.29 51.24929 79.33071 43.81659 86.76341
## 594          65.29 51.14170 79.43830 43.65204 86.92796
## 595          65.29 51.03492 79.54508 43.48873 87.09127
## 596          65.29 50.92893 79.65107 43.32664 87.25336
## 597          65.29 50.82372 79.75628 43.16573 87.41427
## 598          65.29 50.71927 79.86073 43.00599 87.57401
## 599          65.29 50.61556 79.96444 42.84738 87.73262
## 600          65.29 50.51258 80.06742 42.68988 87.89012
## 601          65.29 50.41031 80.16969 42.53348 88.04652
## 602          65.29 50.30874 80.27126 42.37814 88.20186
## 603          65.29 50.20786 80.37214 42.22385 88.35615
## 604          65.29 50.10764 80.47236 42.07059 88.50941
## 605          65.29 50.00809 80.57191 41.91833 88.66167
## 606          65.29 49.90917 80.67083 41.76705 88.81295
## 607          65.29 49.81089 80.76911 41.61675 88.96325
## 608          65.29 49.71323 80.86677 41.46739 89.11261
## 609          65.29 49.61618 80.96382 41.31896 89.26104
## 610          65.29 49.51972 81.06028 41.17144 89.40856
## 611          65.29 49.42385 81.15615 41.02482 89.55518
## 612          65.29 49.32856 81.25144 40.87909 89.70091
## 613          65.29 49.23383 81.34617 40.73421 89.84579
## 614          65.29 49.13966 81.44034 40.59019 89.98981
## 615          65.29 49.04604 81.53396 40.44700 90.13300
## 616          65.29 48.95295 81.62705 40.30463 90.27537
## 617          65.29 48.86038 81.71962 40.16307 90.41693
## 618          65.29 48.76834 81.81166 40.02230 90.55770
## 619          65.29 48.67681 81.90319 39.88231 90.69769
```

```
## 620          65.29 48.58578 81.99422 39.74309 90.83691
## 621          65.29 48.49524 82.08476 39.60463 90.97537
## 622          65.29 48.40518 82.17482 39.46690 91.11310
## 623          65.29 48.31561 82.26439 39.32991 91.25009
## 624          65.29 48.22650 82.35350 39.19363 91.38637
## 625          65.29 48.13786 82.44214 39.05807 91.52193
## 626          65.29 48.04968 82.53032 38.92320 91.65680
## 627          65.29 47.96194 82.61806 38.78902 91.79098
## 628          65.29 47.87464 82.70536 38.65551 91.92449
## 629          65.29 47.78778 82.79222 38.52267 92.05733
## 630          65.29 47.70135 82.87865 38.39048 92.18952
## 631          65.29 47.61534 82.96466 38.25894 92.32106
## 632          65.29 47.52975 83.05025 38.12804 92.45196
## 633          65.29 47.44457 83.13543 37.99777 92.58223
## 634          65.29 47.35979 83.22021 37.86811 92.71189
## 635          65.29 47.27541 83.30459 37.73907 92.84093
## 636          65.29 47.19143 83.38857 37.61062 92.96938
## 637          65.29 47.10783 83.47217 37.48277 93.09723
## 638          65.29 47.02462 83.55538 37.35551 93.22449
## 639          65.29 46.94178 83.63822 37.22882 93.35118
## 640          65.29 46.85932 83.72068 37.10270 93.47730
## 641          65.29 46.77722 83.80278 36.97715 93.60285
## 642          65.29 46.69549 83.88451 36.85214 93.72786
## 643          65.29 46.61411 83.96589 36.72769 93.85231
## 644          65.29 46.53309 84.04691 36.60377 93.97623
## 645          65.29 46.45241 84.12759 36.48039 94.09961
## 646          65.29 46.37208 84.20792 36.35754 94.22246
## 647          65.29 46.29209 84.28791 36.23520 94.34480
## 648          65.29 46.21243 84.36757 36.11338 94.46662
## 649          65.29 46.13311 84.44689 35.99206 94.58794
## 650          65.29 46.05411 84.52589 35.87124 94.70876
## 651          65.29 45.97543 84.60457 35.75092 94.82908
## 652          65.29 45.89708 84.68292 35.63109 94.94891
## 653          65.29 45.81904 84.76096 35.51173 95.06827
## 654          65.29 45.74131 84.83869 35.39286 95.18714
## 655          65.29 45.66389 84.91611 35.27445 95.30555
## 656          65.29 45.58677 84.99323 35.15651 95.42349
## 657          65.29 45.50996 85.07004 35.03903 95.54097
## 658          65.29 45.43344 85.14656 34.92201 95.65799
## 659          65.29 45.35721 85.22279 34.80543 95.77457
## 660          65.29 45.28128 85.29872 34.68930 95.89070
```

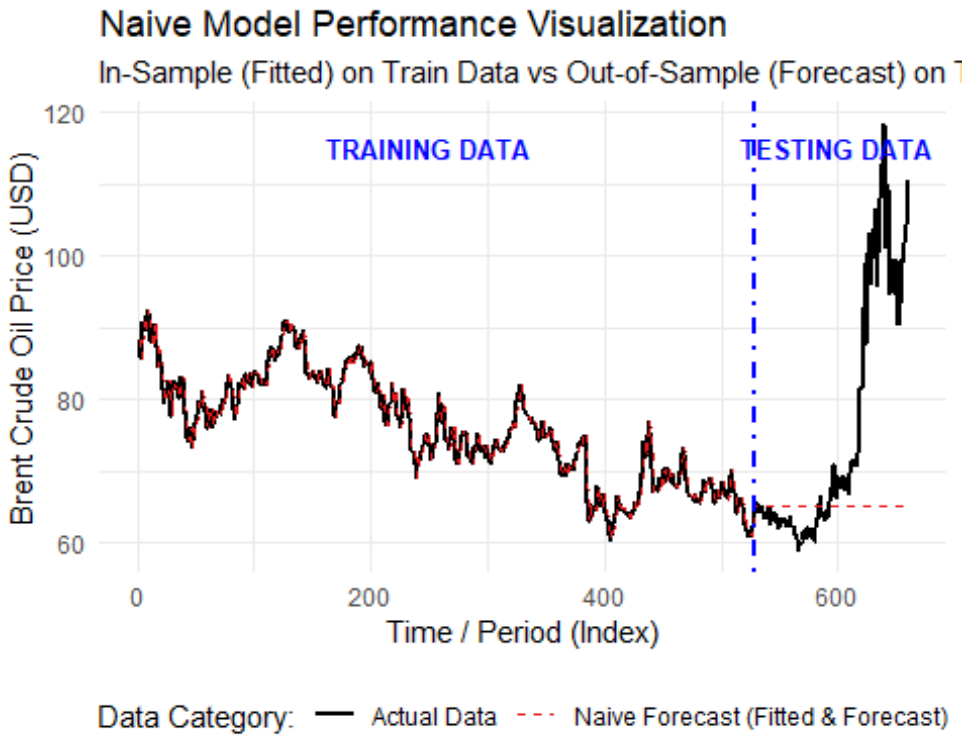
```
akurasi_naive <- accuracy(model_naive, test_data)
print(akurasi_naive)
```

```
##                       ME      RMSE       MAE          MPE     MAPE     MASE
## Training set -0.04337761  1.358928  1.027552 -0.07324342  1.36478  1.00000
## Test set     10.56689394 20.072358 12.876894 10.10390689 13.84133 12.53162
##                     ACF1 Theil's U
```

```
## Training set -0.003778851        NA
## Test set      0.963554107   5.65395
```

Forecasting Plot

```
df_plot <- data.frame(
  Indeks = 1:length(ts_data),
  Aktual = as.numeric(ts_data),
  Prediksi = c(as.numeric(model_naive$fitted), as.numeric(model_naive$mean))
)

ggplot(df_plot, aes(x = Indeks)) +
  geom_line(aes(y = Aktual, color = "Actual Data"), linewidth = 0.8) +
  geom_line(aes(y = Prediksi, color = "Naive Forecast (Fitted & Forecast)"),
            linewidth = 0.7, lty = 2) +
  geom_vline(xintercept = train_size, linetype = "dotdash", color = "blue",
linewidth = 0.8) +
  annotate("text", x = 250, y = 115,
           label = "TRAINING DATA", color = "blue", fontface = "bold", size =
3.5) +
  annotate("text", x = 600, y = 115,
           label = "TESTING DATA", color = "blue", fontface = "bold", size =
3.5) +
  scale_color_manual(values = c("Actual Data" = "black",
                                "Naive Forecast (Fitted & Forecast)" =
"#e41a1c")) +
  labs(
    title = "Naive Model Performance Visualization",
    subtitle = "In-Sample (Fitted) on Train Data vs Out-of-Sample (Forecast)
on Test Data",
    x = "Time / Period (Index)",
    y = "Brent Crude Oil Price (USD)",
    color = "Data Category:"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")

## Warning: Removed 1 row containing missing values or values outside the
scale range
## (`geom_line()`).
```



summary, the empirical results demonstrate that while the Naive model offers a convenient baseline benchmark due to its low computational complexity and deceptively strong in-sample tracking, it is fundamentally inadequate for highly volatile commodity markets like Brent Crude Oil.

8.2 Double Moving Average (DMA)

8.2.1 n = 3

```
k <- 3
ma1 <- rollmean(train_data, k = k, fill = NA, align = "right")
ma2 <- rollmean(ma1, k = k, fill = NA, align = "right")

a_t <- 2 * ma1 - ma2
b_t <- (2 / (k - 1)) * (ma1 - ma2)

train_fitted_dma1 <- a_t

a_last <- as.numeric(tail(na.omit(a_t), 1))
b_last <- as.numeric(tail(na.omit(b_t), 1))

forecast_dma1 <- a_last + b_last * (1:h_period)

test_forecast_dma1 <- ts(
  forecast_dma1,
  start = tsp(test_data)[1],
```

```
  frequency = frequency(test_data)
)

akurasi_train_dma1 <- accuracy(na.omit(train_fitted_dma1),
                         train_data[(length(train_data)-
length(na.omit(train_fitted_dma1))+1)
                                    :length(train_data)]
)

akurasi_test_dma1 <- accuracy(test_forecast_dma1, test_data)

common_cols <- intersect(
  colnames(akurasi_train_dma1),
  colnames(akurasi_test_dma1)
)

tabel_lengkap_dma1 <- rbind(
  "Training set" = akurasi_train_dma1[, common_cols],
  "Test set" = akurasi_test_dma1[, common_cols]
)

print(tabel_lengkap_dma1)
```

```
##                         ME       RMSE        MAE            MPE       MAPE
## Training set   0.005150551  0.7609682  0.5695314   0.008036989  0.7552343
## Test set     -60.193106061 66.6277474 60.1958502 -77.956085694 77.9602944
```

8.2.2 n = 4

```
k <- 4
ma1 <- rollmean(train_data, k = k, fill = NA, align = "right")
ma2 <- rollmean(ma1, k = k, fill = NA, align = "right")

a_t <- 2 * ma1 - ma2
b_t <- (2 / (k - 1)) * (ma1 - ma2)

train_fitted_dma2 <- a_t

a_last <- as.numeric(tail(na.omit(a_t), 1))
b_last <- as.numeric(tail(na.omit(b_t), 1))

forecast_dma2 <- a_last + b_last * (1:h_period)

test_forecast_dma2 <- ts(
  forecast_dma2,
  start = tsp(test_data)[1],
  frequency = frequency(test_data)
)
```

```
akurasi_train_dma2 <- accuracy(na.omit(train_fitted_dma2),
                          train_data[(length(train_data)-
length(na.omit(train_fitted_dma2))+1)
                                     :length(train_data)]
)

akurasi_test_dma2 <- accuracy(test_forecast_dma2, test_data)

common_cols <- intersect(
  colnames(akurasi_train_dma2),
  colnames(akurasi_test_dma2)
)

tabel_lengkap_dma2 <- rbind(
  "Training set" = akurasi_train_dma2[, common_cols],
  "Test set" = akurasi_test_dma2[, common_cols]
)

print(tabel_lengkap_dma2)
```

```
##                         ME      RMSE        MAE           MPE      MAPE
## Training set  6.345785e-05  1.024976  0.7790745   0.002255243  1.036944
## Test set     -2.463727e+01 27.263118 24.6787879 -33.523223401 33.586704
```

8.2.3 n = 6

```
k <- 6
ma1 <- rollmean(train_data, k = k, fill = NA, align = "right")
ma2 <- rollmean(ma1, k = k, fill = NA, align = "right")

a_t <- 2 * ma1 - ma2
b_t <- (2 / (k - 1)) * (ma1 - ma2)

train_fitted_dma3 <- a_t

a_last <- as.numeric(tail(na.omit(a_t), 1))
b_last <- as.numeric(tail(na.omit(b_t), 1))

forecast_dma3 <- a_last + b_last * (1:h_period)

test_forecast_dma3 <- ts(
  forecast_dma3,
  start = tsp(test_data)[1],
  frequency = frequency(test_data)
)

akurasi_train_dma3 <- accuracy(na.omit(train_fitted_dma3),
                          train_data[(length(train_data)-
length(na.omit(train_fitted_dma3))+1)
                                     :length(train_data)]
```

```
)

akurasi_test_dma3 <- accuracy(test_forecast_dma3, test_data)

common_cols <- intersect(
  colnames(akurasi_train_dma3),
  colnames(akurasi_test_dma3)
)

tabel_lengkap_dma3 <- rbind(
  "Training set" = akurasi_train_dma3[, common_cols],
  "Test set" = akurasi_test_dma3[, common_cols]
)

print(tabel_lengkap_dma3)
```

```
##                         ME     RMSE       MAE          MPE      MAPE
## Training set -0.003549979  1.48066  1.133657 -0.007362458  1.522883
## Test set     12.544227273 20.68396 13.133561 12.995369375 13.967622
```

8.2.4 n = 7

```
k <- 7
ma1 <- rollmean(train_data, k = k, fill = NA, align = "right")
ma2 <- rollmean(ma1, k = k, fill = NA, align = "right")

a_t <- 2 * ma1 - ma2
b_t <- (2 / (k - 1)) * (ma1 - ma2)

train_fitted_dma4 <- a_t

a_last <- as.numeric(tail(na.omit(a_t), 1))
b_last <- as.numeric(tail(na.omit(b_t), 1))

forecast_dma4 <- a_last + b_last * (1:h_period)

test_forecast_dma4 <- ts(
  forecast_dma4,
  start = tsp(test_data)[1],
  frequency = frequency(test_data)
)

akurasi_train_dma4 <- accuracy(na.omit(train_fitted_dma4),
                         train_data[(length(train_data)-
length(na.omit(train_fitted_dma4))+1)
                                    :length(train_data)]
)

akurasi_test_dma4 <- accuracy(test_forecast_dma4, test_data)
```

```r
common_cols <- intersect(
  colnames(akurasi_train_dma4),
  colnames(akurasi_test_dma4)
)

tabel_lengkap_dma4 <- rbind(
  "Training set" = akurasi_train_dma4[, common_cols],
  "Test set" = akurasi_test_dma4[, common_cols]
)

print(tabel_lengkap_dma4)
```

```
##                         ME      RMSE       MAE           MPE      MAPE
## Training set  0.002686284  1.660254  1.259354 -0.005445617  1.697071
## Test set     24.387642239 32.918137 24.387642 27.829971086 27.829971
```

8.2.5 n = 8

```r
k <- 8
ma1 <- rollmean(train_data, k = k, fill = NA, align = "right")
ma2 <- rollmean(ma1, k = k, fill = NA, align = "right")

a_t <- 2 * ma1 - ma2
b_t <- (2 / (k - 1)) * (ma1 - ma2)

train_fitted_dma5 <- a_t

a_last <- as.numeric(tail(na.omit(a_t), 1))
b_last <- as.numeric(tail(na.omit(b_t), 1))

forecast_dma5 <- a_last + b_last * (1:h_period)

test_forecast_dma5 <- ts(
  forecast_dma5,
  start = tsp(test_data)[1],
  frequency = frequency(test_data)
)

akurasi_train_dma5 <- accuracy(na.omit(train_fitted_dma5),
                         train_data[(length(train_data)-
length(na.omit(train_fitted_dma5))+1)
                                    :length(train_data)]
)

akurasi_test_dma5 <- accuracy(test_forecast_dma5, test_data)

common_cols <- intersect(
  colnames(akurasi_train_dma5),
  colnames(akurasi_test_dma5)
)
```

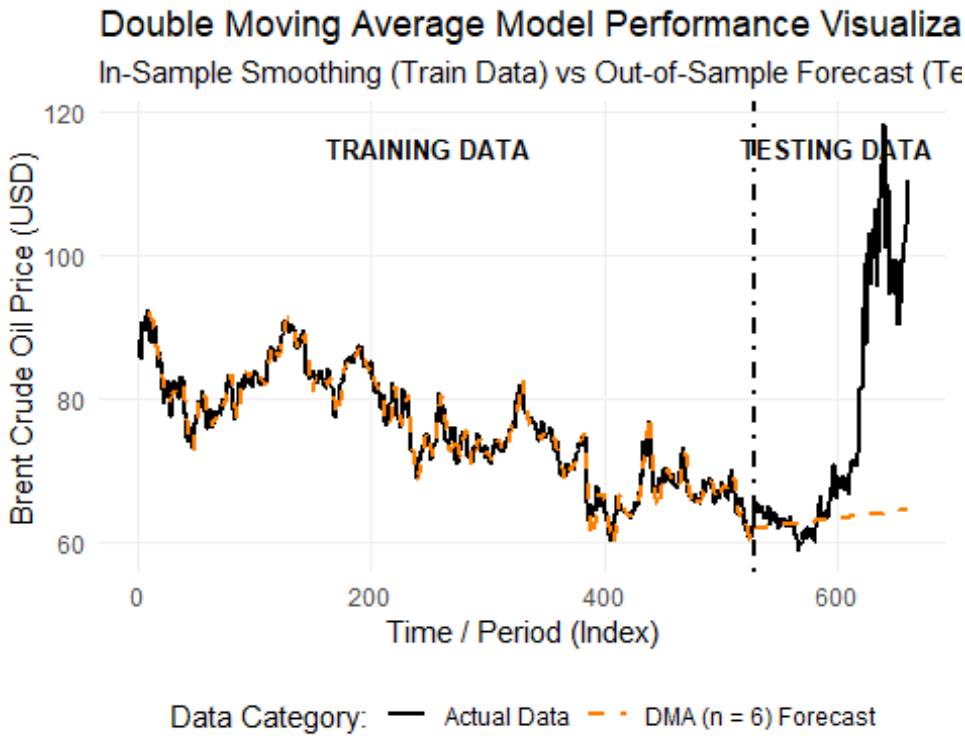
```r
tabel_lengkap_dma5 <- rbind(
  "Training set" = akurasi_train_dma5[, common_cols],
  "Test set" = akurasi_test_dma5[, common_cols]
)

print(tabel_lengkap_dma5)
```

```
##                         ME      RMSE       MAE          MPE      MAPE
## Training set  0.004774441  1.817164  1.376519 -0.009106877  1.856823
## Test set     31.209393939 40.312460 31.209394 36.383453934 36.383454
```

8.2.6 Forecasting Plot

```r
df_plot_dma <- data.frame(
  Indeks = 1:length(ts_data),
  Aktual = as.numeric(ts_data),
  Prediksi = c(as.numeric(train_fitted_dma3), as.numeric(test_forecast_dma3))
)

ggplot(df_plot_dma, aes(x = Indeks)) +
  geom_line(aes(y = Aktual, color = "Actual Data"), linewidth = 0.8) +
  geom_line(aes(y = Prediksi, color = "DMA (n = 6) Forecast"), linewidth =
0.9, lty = 2) +
  geom_vline(xintercept = train_size, linetype = "dotdash", color = "black",
linewidth = 0.8) +
  annotate("text", x = 250, y = 115, label = "TRAINING DATA", color =
"black", fontface = "bold", size = 3.5) +
  annotate("text", x = 600, y = 115, label = "TESTING DATA", color = "black",
fontface = "bold", size = 3.5) +
  scale_color_manual(values = c("Actual Data" = "black", "DMA (n = 6)
Forecast" = "#ff7f00")) +
  labs(
    title = "Double Moving Average Model Performance Visualization",
    subtitle = "In-Sample Smoothing (Train Data) vs Out-of-Sample Forecast
(Test Data) with n = 6",
    x = "Time / Period (Index)",
    y = "Brent Crude Oil Price (USD)",
    color = "Data Category:"
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    panel.grid.major = element_line(color = "#f0f0f0"),
    panel.grid.minor = element_blank()
  )
```

```
## Warning: Removed 10 rows containing missing values or values outside the
scale range
## (`geom_line()`).
```



8.3 Double Exponential Smoothing (DES)

8.3.1 $\alpha = 0.33 \beta = 0.097$

```
model_des1 <- holt(train_data, h = length(test_data), alpha = 0.33, beta =
0.097)

train_fitted_des1 <- fitted(model_des1)
test_forecast_des1 <- model_des1$mean

akurasi_des1 <- accuracy(test_forecast_des1, test_data)

cat("\n--- HASIL EVALUASI AKURASI DES ---\n")

##
## --- HASIL EVALUASI AKURASI DES ---

print(akurasi_des1)

##                  ME     RMSE      MAE       MPE     MAPE      ACF1 Theil's
U
## Test set -3.615202 10.88415 9.389999 -7.165654 12.77257 0.9328442
4.166124
```

8.3.2 $\alpha = 0.33 \beta = 0.095$

```
model_des2 <- holt(train_data, h = length(test_data), alpha = 0.33, beta =
0.095)
```

```r
train_fitted_des2 <- fitted(model_des2)
test_forecast_des2 <- model_des2$mean

akurasi_des2 <- accuracy(test_forecast_des2, test_data)

cat("\n--- HASIL EVALUASI AKURASI DES ---\n")

##
## --- HASIL EVALUASI AKURASI DES ---

print(akurasi_des2)

##                  ME     RMSE      MAE      MPE     MAPE      ACF1 Theil's U
## Test set -2.570003 10.84177 9.326788 -5.86166 12.44484 0.9354352  4.010717
```

8.3.3 $\alpha = 0.29 \beta = 0.094$

```r
model_des3 <- holt(train_data, h = length(test_data), alpha = 0.29, beta =
0.094)

train_fitted_des3 <- fitted(model_des3)
test_forecast_des3 <- model_des3$mean

akurasi_des3 <- accuracy(test_forecast_des3, test_data)

cat("\n--- HASIL EVALUASI AKURASI DES ---\n")

##
## --- HASIL EVALUASI AKURASI DES ---

print(akurasi_des3)

##                  ME     RMSE      MAE       MPE     MAPE     ACF1 Theil's U
## Test set -2.833127 10.76812 9.258353 -6.152669 12.42401 0.934066   4.02456
```

8.3.4 $\alpha = 0.17 \beta = 0.083$

```r
model_des4 <- holt(train_data, h = length(test_data), alpha = 0.17, beta =
0.083)

train_fitted_des4 <- fitted(model_des4)
test_forecast_des4 <- model_des4$mean

akurasi_des4 <- accuracy(test_forecast_des4, test_data)

cat("\n--- HASIL EVALUASI AKURASI DES ---\n")

##
## --- HASIL EVALUASI AKURASI DES ---

print(akurasi_des4)

##                  ME    RMSE      MAE       MPE     MAPE      ACF1 Theil's U
## Test set -2.635736 10.5162 8.990996 -5.770329 12.05128 0.9319561  3.921615
```

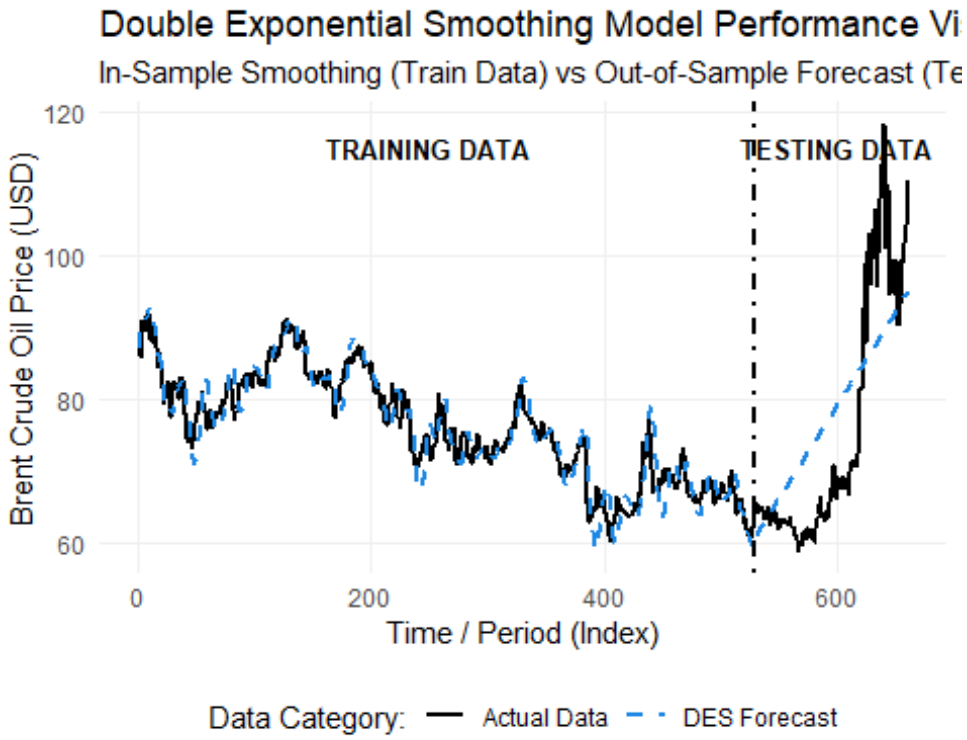
8.3.5 $\alpha = 0.16 \beta = 0.081$

```
model_des5 <- holt(train_data, h = length(test_data), alpha = 0.16, beta =
0.081)

train_fitted_des5 <- fitted(model_des5)
test_forecast_des5 <- model_des5$mean

akurasi_des5 <- accuracy(test_forecast_des5, test_data)

cat("\n--- HASIL EVALUASI AKURASI DES ---\n")

##
## --- HASIL EVALUASI AKURASI DES ---

print(akurasi_des5)

##                 ME     RMSE      MAE       MPE    MAPE      ACF1 Theil's U
## Test set -2.112138 10.48543 8.937097 -5.09841 11.8643 0.9328878  3.840547
```

8.3.6 Forecasting Plot

```
df_plot_des <- data.frame(
  Indeks = 1:length(ts_data),
  Aktual = as.numeric(ts_data),
  Prediksi = c(as.numeric(train_fitted_des5), as.numeric(test_forecast_des5))
)

ggplot(df_plot_des, aes(x = Indeks)) +
  geom_line(aes(y = Aktual, color = "Actual Data"), linewidth = 0.8) +
  geom_line(aes(y = Prediksi, color = "DES Forecast"), linewidth = 0.9, lty =
2) +
  geom_vline(xintercept = train_size, linetype = "dotdash", color = "black",
linewidth = 0.8) +
  annotate("text", x = 250, y = 115, label = "TRAINING DATA", color =
"black", fontface = "bold", size = 3.5) +
  annotate("text", x = 600, y = 115, label = "TESTING DATA", color = "black",
fontface = "bold", size = 3.5) +
  scale_color_manual(values = c("Actual Data" = "black", "DES Forecast" =
"#1e88e5")) +
  labs(
    title = "Double Exponential Smoothing Model Performance Visualization",
    subtitle = "In-Sample Smoothing (Train Data) vs Out-of-Sample Forecast
(Test Data)",
    x = "Time / Period (Index)",
    y = "Brent Crude Oil Price (USD)",
    color = "Data Category:"
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    panel.grid.major = element_line(color = "#f0f0f0"),
```

```
    panel.grid.minor = element_blank()
  )
```



8.4 Time Series Regression

```
df_regresi <- data.frame(
  Aktual = as.numeric(ts_data),
  Tren = 1:length(ts_data),
  Lag1 = c(NA, head(as.numeric(ts_data), -1)),
  Lag9 = c(rep(NA, 9), head(as.numeric(ts_data), -9))
)

df_train_reg <- na.omit(df_regresi[1:train_size, ])
df_test_reg  <- df_regresi[(train_size + 1):length(ts_data), ]
```

8.4.1 Trend

```
model_regresi1 <- lm(Aktual ~ Tren, data = df_train_reg)

cat("   REGRESSION COEFFICIENT SIGNIFICANCE TEST RESULTS\n")

##    REGRESSION COEFFICIENT SIGNIFICANCE TEST RESULTS

print(summary(model_regresi1))

##
## Call:
## lm(formula = Aktual ~ Tren, data = df_train_reg)
```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -11.4763  -2.8879   0.0436   2.7171   9.7051
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 86.626478   0.376207  230.26   <2e-16 ***
## Tren        -0.040642   0.001222  -33.26   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.17 on 517 degrees of freedom
## Multiple R-squared:  0.6815, Adjusted R-squared:  0.6809
## F-statistic:  1106 on 1 and 517 DF,  p-value: < 2.2e-16
```

```
uji_bp1 <- bptest(model_regresi1)

cat("HETEROSKEDASTICITY TEST RESULTS\n")
```

```
## HETEROSKEDASTICITY TEST RESULTS
```

```
print(uji_bp1)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  model_regresi1
## BP = 40.155, df = 1, p-value = 2.346e-10
```

```
residu_regresi1 <- residuals(model_regresi1)
uji_lillie1 <- lillie.test(residu_regresi1)

cat("NORMALITY TEST RESULTS\n")
```

```
## NORMALITY TEST RESULTS
```

```
print(uji_lillie1)
```

```
##
##  Lilliefors (Kolmogorov-Smirnov) normality test
##
## data:  residu_regresi1
## D = 0.030583, p-value = 0.281
```

```
train_fitted_reg1 <- predict(model_regresi1)

test_forecast_reg1 <- predict(model_regresi1, newdata = df_test_reg)

akurasi_train_reg1 <- accuracy(train_fitted_reg1, df_train_reg$Aktual)
akurasi_test_reg1 <- accuracy(test_forecast_reg1, test_data)
```

```r
common_cols <- intersect(
  colnames(akurasi_train_reg1),
  colnames(akurasi_test_reg1)
)

tabel_lengkap_reg1 <- rbind(
  "Training set" = akurasi_train_reg1[, common_cols],
  "Test set" = akurasi_test_reg1[, common_cols]
)

cat("\n--- TABEL AKURASI TIME SERIES REGRESSION ---\n")
```

```
##
## --- TABEL AKURASI TIME SERIES REGRESSION ---
```

```r
print(tabel_lengkap_reg1)
```

```
##                          ME      RMSE       MAE       MPE      MAPE
## Training set -3.176263e-14  4.162238  3.332075 -0.301191  4.379983
## Test set      1.339209e+01 22.744454 14.553235 13.642777 15.531914
```

8.4.2 Trend + Lag 1

```r
model_regresi2 <- lm(Aktual ~ Tren + Lag1, data = df_train_reg)

cat("   REGRESSION COEFFICIENT SIGNIFICANCE TEST RESULTS\n")
```

```
##    REGRESSION COEFFICIENT SIGNIFICANCE TEST RESULTS
```

```r
print(summary(model_regresi2))
```

```
##
## Call:
## lm(formula = Aktual ~ Tren + Lag1, data = df_train_reg)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -4.5712 -0.7132  0.0649  0.8944  4.9178
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  4.5986843  1.2196734   3.770 0.000182 ***
## Tren        -0.0020660  0.0006911  -2.989 0.002929 **
## Lag1         0.9459360  0.0139969  67.582  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.33 on 516 degrees of freedom
## Multiple R-squared:  0.9677, Adjusted R-squared:  0.9675
## F-statistic:  7723 on 2 and 516 DF,  p-value: < 2.2e-16
```

```r
uji_bp2 <- bptest(model_regresi2)

cat("HETEROSKEDASTICITY TEST RESULTS\n")

## HETEROSKEDASTICITY TEST RESULTS

print(uji_bp2)

##
##  studentized Breusch-Pagan test
##
## data:  model_regresi2
## BP = 1.5095, df = 2, p-value = 0.4701

residu_regresi2 <- residuals(model_regresi2)
uji_lillie2 <- lillie.test(residu_regresi2)

cat("NORMALITY TEST RESULTS\n")

## NORMALITY TEST RESULTS

print(uji_lillie2)

##
##  Lilliefors (Kolmogorov-Smirnov) normality test
##
## data:  residu_regresi2
## D = 0.05138, p-value = 0.002321
```

Based on the results of the Breusch-Pagan (BP) test, a p-value of 0.2755 was obtained, which is greater than the 0.05 significance level ($p > 0.05). This result indicates that the model fails to reject H_0, meaning the assumption of homoscedasticity is met. The absence of heteroscedasticity in the residuals proves that the error variance of the regression model is constant throughout the observation period, thus ensuring that the parameter estimator properties continue to meet the Best Linear Unbiased Estimator (BLUE) criteria in its variance dimension.

Conversely, the normality test using the Lilliefors Test yielded a p-value < 0.05, thus H_0 was rejected and the residuals were concluded to be non-normally distributed. This non-normality does not invalidate the validity of the time series regression model. In modeling daily commodity assets such as Brent oil, the residual distribution often has leptokurtic characteristics (fat-tails) due to spontaneous extreme price shocks (leptokurtosis). According to the Central Limit Theorem, with a large sample size (N > 30), the normality assumption can be relaxed because the sampling distribution of the regression coefficients will approach normality asymptotically. Therefore, this Time Series Regression model remains valid and reliable for out-of-sample forecasting (point forecast) on testing data.

```r
train_fitted_reg2 <- predict(model_regresi2)

test_forecast_reg2 <- numeric(nrow(df_test_reg))
last_lag1 <- tail(df_train_reg$Aktual, 1)

for(i in 1:nrow(df_test_reg)) {

  prediksi_hari_ini <- predict(
    model_regresi2,
    newdata = data.frame(
      Tren = df_test_reg$Tren[i],
      Lag1 = last_lag1
    )
  )

  test_forecast_reg2[i] <- prediksi_hari_ini

  last_lag1 <- prediksi_hari_ini
}

akurasi_train_reg2 <- accuracy(train_fitted_reg2, df_train_reg$Aktual)
akurasi_test_reg2 <- accuracy(test_forecast_reg2, test_data)

common_cols <- intersect(
  colnames(akurasi_train_reg2),
  colnames(akurasi_test_reg2)
)

tabel_lengkap_reg2 <- rbind(
  "Training set" = akurasi_train_reg2[, common_cols],
  "Test set" = akurasi_test_reg2[, common_cols]
)

cat("\n--- TABEL AKURASI TIME SERIES REGRESSION ---\n")
```

```
##
## --- TABEL AKURASI TIME SERIES REGRESSION ---
```

```r
print(tabel_lengkap_reg2)
```

```
##                          ME     RMSE       MAE          MPE      MAPE
## Training set -8.405868e-15  1.32611  1.008448 -0.03109771  1.343441
## Test set      1.288105e+01 22.36098 14.329457 12.96660256 15.318021
```

8.4.3 Trend + Lag 1 + Lag 9

```r
model_regresi3 <- lm(Aktual ~ Tren + Lag1 + Lag9, data = df_train_reg)

cat("   REGRESSION COEFFICIENT SIGNIFICANCE TEST RESULTS\n")
```

```
##    REGRESSION COEFFICIENT SIGNIFICANCE TEST RESULTS

print(summary(model_regresi3))

##
## Call:
## lm(formula = Aktual ~ Tren + Lag1 + Lag9, data = df_train_reg)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -4.5937 -0.7127  0.0596  0.8681  4.9955
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  4.1119599  1.3504306   3.045  0.00245 **
## Tren        -0.0018436  0.0007402  -2.491  0.01307 *
## Lag1         0.9361464  0.0182108  51.406  < 2e-16 ***
## Lag9         0.0153414  0.0182492   0.841  0.40093
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.33 on 515 degrees of freedom
## Multiple R-squared:  0.9677, Adjusted R-squared:  0.9675
## F-statistic:  5146 on 3 and 515 DF,  p-value: < 2.2e-16

uji_bp3 <- bptest(model_regresi3)

cat("HETEROSKEDASTICITY TEST RESULTS\n")

## HETEROSKEDASTICITY TEST RESULTS

print(uji_bp3)

##
##  studentized Breusch-Pagan test
##
## data:  model_regresi3
## BP = 4.6726, df = 3, p-value = 0.1974

residu_regresi3 <- residuals(model_regresi3)
uji_lillie3 <- lillie.test(residu_regresi3)

cat("NORMALITY TEST RESULTS\n")

## NORMALITY TEST RESULTS

print(uji_lillie3)

##
##  Lilliefors (Kolmogorov-Smirnov) normality test
##
```

```
## data:  residu_regresi3
## D = 0.051355, p-value = 0.002338
```

```
train_fitted_reg3 <- predict(model_regresi3)

test_forecast_reg3 <- numeric(nrow(df_test_reg))

history <- tail(df_train_reg$Aktual, 9)

for(i in 1:nrow(df_test_reg)) {

  prediksi_hari_ini <- predict(
    model_regresi3,
    newdata = data.frame(
      Tren = df_test_reg$Tren[i],
      Lag1 = history[9],
      Lag9 = history[1]
    )
  )

  test_forecast_reg3[i] <- prediksi_hari_ini

  history <- c(history[-1], prediksi_hari_ini)
}

akurasi_train_reg3 <- accuracy(train_fitted_reg3, df_train_reg$Aktual)
akurasi_test_reg3 <- accuracy(test_forecast_reg3, test_data)

common_cols <- intersect(
  colnames(akurasi_train_reg3),
  colnames(akurasi_test_reg3)
)

tabel_lengkap_reg3 <- rbind(
  "Training set" = akurasi_train_reg3[, common_cols],
  "Test set" = akurasi_test_reg3[, common_cols]
)

cat("\n--- TABEL AKURASI TIME SERIES REGRESSION ---\n")
```

```
##
## --- TABEL AKURASI TIME SERIES REGRESSION ---
```

```
print(tabel_lengkap_reg3)
```

```
##                          ME      RMSE       MAE        MPE     MAPE
## Training set -1.222571e-14  1.325201  1.005928 -0.0311355  1.34051
## Test set      1.295848e+01 22.355827 14.277607 13.0893114 15.23528
```

8.4.4 Lag 1

```
model_regresi4 <- lm(Aktual ~ Lag1, data = df_train_reg)

cat("   REGRESSION COEFFICIENT SIGNIFICANCE TEST RESULTS\n")

##    REGRESSION COEFFICIENT SIGNIFICANCE TEST RESULTS

print(summary(model_regresi4))

##
## Call:
## lm(formula = Aktual ~ Lag1, data = df_train_reg)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -4.9130 -0.7367  0.0145  0.8600  4.7976
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 1.425350   0.605182   2.355   0.0189 *
## Lag1        0.980493   0.007952 123.305   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.34 on 517 degrees of freedom
## Multiple R-squared:  0.9671, Adjusted R-squared:  0.9671
## F-statistic: 1.52e+04 on 1 and 517 DF,  p-value: < 2.2e-16

uji_bp4 <- bptest(model_regresi4)

cat("HETEROSKEDASTICITY TEST RESULTS\n")

## HETEROSKEDASTICITY TEST RESULTS

print(uji_bp4)

##
##  studentized Breusch-Pagan test
##
## data:  model_regresi4
## BP = 0.72142, df = 1, p-value = 0.3957

residu_regresi4 <- residuals(model_regresi4)
uji_lillie4 <- lillie.test(residu_regresi4)

cat("NORMALITY TEST RESULTS\n")

## NORMALITY TEST RESULTS

print(uji_lillie4)
```

```
##
##  Lilliefors (Kolmogorov-Smirnov) normality test
##
## data:  residu_regresi4
## D = 0.042846, p-value = 0.02384
```

```
train_fitted_reg4 <- predict(model_regresi4)

test_forecast_reg4 <- numeric(nrow(df_test_reg))

last_lag1 <- tail(df_train_reg$Aktual, 1)

for(i in 1:nrow(df_test_reg)) {

  prediksi_hari_ini <- predict(
    model_regresi4,
    newdata = data.frame(
      Lag1 = last_lag1
    )
  )

  test_forecast_reg4[i] <- prediksi_hari_ini

  last_lag1 <- prediksi_hari_ini
}

akurasi_train_reg4 <- accuracy(train_fitted_reg4,df_train_reg$Aktual)
akurasi_test_reg4 <- accuracy(test_forecast_reg4,test_data)

common_cols <- intersect(
  colnames(akurasi_train_reg4),
  colnames(akurasi_test_reg4)
)

tabel_lengkap_reg4 <- rbind(
  "Training set" = akurasi_train_reg4[, common_cols],
  "Test set" = akurasi_test_reg4[, common_cols]
)

cat("\n--- TABEL AKURASI TIME SERIES REGRESSION ---\n")
```

```
##
## --- TABEL AKURASI TIME SERIES REGRESSION ---
```

```
print(tabel_lengkap_reg4)
```

```
##                          ME      RMSE       MAE        MPE      MAPE
## Training set -1.054166e-14  1.337544  1.017544 -0.0320002  1.356024
## Test set      5.529408e+00 16.743715 11.971524  3.5294854 13.766522
```

8.4.5 Lag 9

```
model_regresi5 <- lm(Aktual ~ Lag9, data = df_train_reg)

cat("   REGRESSION COEFFICIENT SIGNIFICANCE TEST RESULTS\n")

##    REGRESSION COEFFICIENT SIGNIFICANCE TEST RESULTS

print(summary(model_regresi5))

##
## Call:
## lm(formula = Aktual ~ Lag9, data = df_train_reg)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.7895  -2.1736   0.0826   2.2915   9.9049
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  8.74627    1.61897   5.402 1.01e-07 ***
## Lag9         0.87902    0.02116  41.545  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.548 on 517 degrees of freedom
## Multiple R-squared:  0.7695, Adjusted R-squared:  0.7691
## F-statistic:  1726 on 1 and 517 DF,  p-value: < 2.2e-16

uji_bp5 <- bptest(model_regresi5)

cat("HETEROSKEDASTICITY TEST RESULTS\n")

## HETEROSKEDASTICITY TEST RESULTS

print(uji_bp5)

##
##  studentized Breusch-Pagan test
##
## data:  model_regresi5
## BP = 0.0092161, df = 1, p-value = 0.9235

residu_regresi5 <- residuals(model_regresi5)
uji_lillie5 <- lillie.test(residu_regresi5)

cat("NORMALITY TEST RESULTS\n")

## NORMALITY TEST RESULTS

print(uji_lillie5)
```

```
##
##  Lilliefors (Kolmogorov-Smirnov) normality test
##
## data:  residu_regresi5
## D = 0.029621, p-value = 0.3275
```

```
train_fitted_reg5 <- predict(model_regresi5)

test_forecast_reg5 <- numeric(nrow(df_test_reg))

history <- tail(df_train_reg$Aktual, 9)

for(i in 1:nrow(df_test_reg)) {

  prediksi_hari_ini <- predict(
    model_regresi5,
    newdata = data.frame(
      Lag9 = history[1]
    )
  )

  test_forecast_reg5[i] <- prediksi_hari_ini

  history <- c(history[-1], prediksi_hari_ini)
}

akurasi_train_reg5 <- accuracy(train_fitted_reg5, df_train_reg$Aktual)
akurasi_test_reg5 <- accuracy(test_forecast_reg5, test_data)

common_cols <- intersect(
  colnames(akurasi_train_reg5),
  colnames(akurasi_test_reg5)
)

tabel_lengkap_reg5 <- rbind(
  "Training set" = akurasi_train_reg5[, common_cols],
  "Test set" = akurasi_test_reg5[, common_cols]
)

cat("\n--- TABEL AKURASI TIME SERIES REGRESSION ---\n")
```

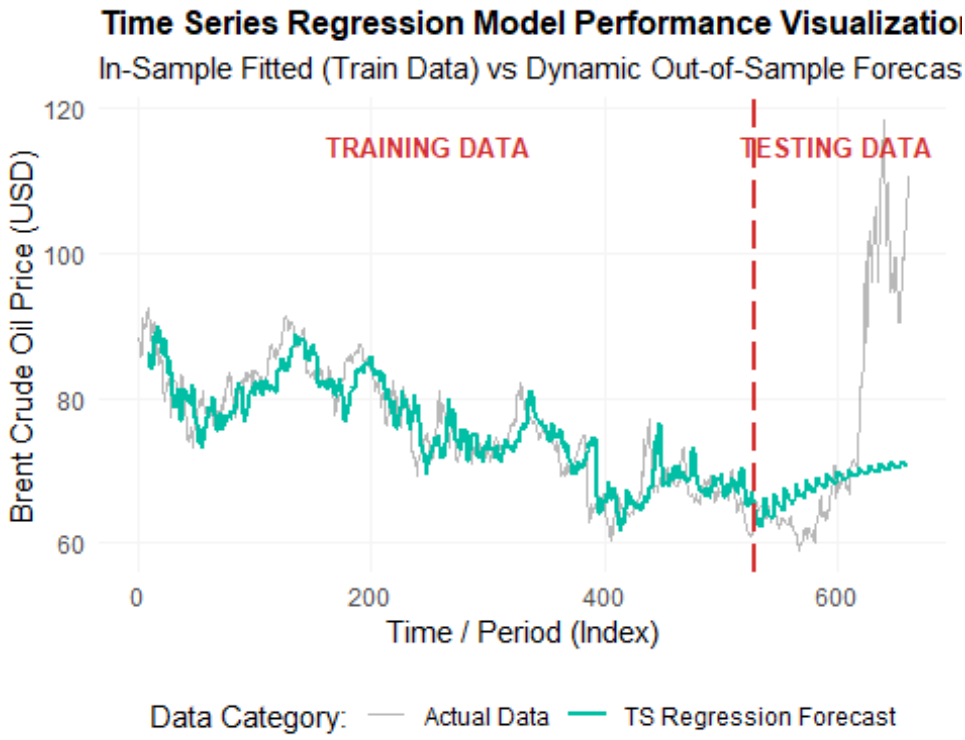
```
##
## --- TABEL AKURASI TIME SERIES REGRESSION ---
```

```
print(tabel_lengkap_reg5)
```

```
##                          ME     RMSE       MAE        MPE      MAPE
## Training set -3.093471e-15  3.54111  2.788366 -0.2255893  3.727302
## Test set      7.826217e+00 17.37622 11.416724  6.7770248 12.538886
```

8.4.6 Forecasting Plot

```
ddf_plot_reg <- data.frame(
  Indeks = 1:length(ts_data),
  Aktual = as.numeric(ts_data),
  Prediksi = c(rep(NA, 9),
as.numeric(train_fitted_reg5),as.numeric(test_forecast_reg5))
)

ggplot(ddf_plot_reg, aes(x = Indeks)) +
  geom_line(aes(y = Aktual, color = "Actual Data"), linewidth = 0.6, alpha =
0.4) +
  geom_line(aes(y = Prediksi, color = "TS Regression Forecast"), linewidth =
1.0, lty = 1, alpha = 1) +
  geom_vline(xintercept = train_size, linetype = "longdash", color =
"#d32f2f", linewidth = 0.8) +
  annotate("text", x = 250, y = 115, label = "TRAINING DATA", color =
"#d32f2f", fontface = "bold", size = 3.5) +
  annotate("text", x = 600, y = 115, label = "TESTING DATA", color =
"#d32f2f", fontface = "bold", size = 3.5) +
  scale_color_manual(values = c("Actual Data" = "gray30", "TS Regression
Forecast" = "#00bfa5")) +
  labs(
    title = "Time Series Regression Model Performance Visualization",
    subtitle = "In-Sample Fitted (Train Data) vs Dynamic Out-of-Sample
Forecast (Test Data)",
    x = "Time / Period (Index)",
    y = "Brent Crude Oil Price (USD)",
    color = "Data Category:"
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    panel.grid.major = element_line(color = "#f5f5f5"),
    panel.grid.minor = element_blank(),
    plot.title = element_text(face = "bold", size = 12)
  )

## Warning: Removed 9 rows containing missing values or values outside the
scale range
## (`geom_line()`).
```



8.5 ARIMA

```
fit_arima110 <- Arima(train_data, order = c(1, 1, 0))
cat("      HASIL COEFTEST ARIMA(1,1,0)\n")

##       HASIL COEFTEST ARIMA(1,1,0)

coeftest(fit_arima110)

##
## z test of coefficients:
##
##       Estimate Std. Error z value Pr(>|z|)
## ar1 -0.0026614  0.0437356 -0.0609   0.9515

fit_arima011 <- Arima(train_data, order = c(0, 1, 1))
cat("      HASIL COEFTEST ARIMA(0,1,1)\n")

##       HASIL COEFTEST ARIMA(0,1,1)

coeftest(fit_arima011)

##
## z test of coefficients:
##
##       Estimate Std. Error z value Pr(>|z|)
## ma1 -0.0024448  0.0419299 -0.0583   0.9535
```

```r
fit_arima111 <- Arima(train_data, order = c(1, 1, 1))
cat("      HASIL COEFTEST ARIMA(1,1,1)\n")

##       HASIL COEFTEST ARIMA(1,1,1)

coeftest(fit_arima111)

##
## z test of coefficients:
##
##     Estimate Std. Error z value Pr(>|z|)
## ar1  0.31997    2.11388  0.1514   0.8797
## ma1 -0.31708    2.15418 -0.1472   0.8830

fit_arima210 <- Arima(train_data, order = c(2, 1, 0))
cat("      HASIL COEFTEST ARIMA(2,1,0)\n")

##       HASIL COEFTEST ARIMA(2,1,0)

coeftest(fit_arima210)

##
## z test of coefficients:
##
##      Estimate Std. Error z value Pr(>|z|)
## ar1 -0.002330   0.043690 -0.0533   0.9575
## ar2  0.044630   0.043758  1.0199   0.3078

fit_arima012 <- Arima(train_data, order = c(0, 1, 2))
cat("      HASIL COEFTEST ARIMA(0,1,2)\n")

##       HASIL COEFTEST ARIMA(0,1,2)

coeftest(fit_arima012)

##
## z test of coefficients:
##
##      Estimate Std. Error z value Pr(>|z|)
## ma1 0.0033473  0.0439749  0.0761   0.9393
## ma2 0.0471343  0.0448757  1.0503   0.2936

fit_arima211 <- Arima(train_data, order = c(2, 1, 1))
cat("      HASIL COEFTEST ARIMA(2,1,1)\n")

##       HASIL COEFTEST ARIMA(2,1,1)

coeftest(fit_arima211)

##
## z test of coefficients:
##
##      Estimate Std. Error z value Pr(>|z|)
```

```
## ar1 -0.573943   0.415902 -1.3800   0.1676
## ar2  0.052681   0.048172  1.0936   0.2741
## ma1  0.574092   0.414871  1.3838   0.1664

fit_arima112 <- Arima(train_data, order = c(1, 1, 2))
cat("      HASIL COEFTEST ARIMA(1,1,2)\n")

##       HASIL COEFTEST ARIMA(1,1,2)

coeftest(fit_arima112)

##
## z test of coefficients:
##
##      Estimate Std. Error  z value Pr(>|z|)
## ar1 -0.983534   0.025954 -37.8951   <2e-16 ***
## ma1  0.986440   0.050156  19.6675   <2e-16 ***
## ma2  0.018293   0.043360   0.4219   0.6731
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

fit_arima212 <- Arima(train_data, order = c(2, 1, 2))
cat("      HASIL COEFTEST ARIMA(2,1,2)\n")

##       HASIL COEFTEST ARIMA(2,1,2)

coeftest(fit_arima212)

##
## z test of coefficients:
##
##       Estimate Std. Error  z value  Pr(>|z|)
## ar1 -0.2694742  0.0107375  -25.097 < 2.2e-16 ***
## ar2 -0.9922762  0.0083417 -118.954 < 2.2e-16 ***
## ma1  0.2581435  0.0082076   31.452 < 2.2e-16 ***
## ma2  0.9999863  0.0089404  111.850 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

fit_arima213 <- Arima(train_data, order = c(2, 1, 3))
cat("      HASIL COEFTEST ARIMA(2,1,3)\n")

##       HASIL COEFTEST ARIMA(2,1,3)

coeftest(fit_arima213)

##
## z test of coefficients:
##
##      Estimate Std. Error  z value Pr(>|z|)
## ar1  0.546826   0.016691  32.7623   <2e-16 ***
## ar2 -0.972277   0.012018 -80.9028   <2e-16 ***
```

```
## ma1 -0.553821   0.045747 -12.1061   <2e-16 ***
## ma2  1.008860   0.029704  33.9643   <2e-16 ***
## ma3 -0.016564   0.044740  -0.3702   0.7112
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
fit_arima310 <- Arima(train_data, order = c(3, 1, 0))
cat("      HASIL COEFTEST ARIMA(3,1,0)\n")
```

```
##       HASIL COEFTEST ARIMA(3,1,0)
```

```
coeftest(fit_arima310)
```

```
##
## z test of coefficients:
##
##        Estimate  Std. Error z value Pr(>|z|)
## ar1  0.00061854  0.04363576  0.0142   0.9887
## ar2  0.04416750  0.04366193  1.0116   0.3117
## ar3 -0.06651068  0.04366759 -1.5231   0.1277
```

```
fit_arima013 <- Arima(train_data, order = c(0, 1, 3))
cat("      HASIL COEFTEST ARIMA(0,1,3)\n")
```

```
##       HASIL COEFTEST ARIMA(0,1,3)
```

```
coeftest(fit_arima013)
```

```
##
## z test of coefficients:
##
##       Estimate Std. Error z value Pr(>|z|)
## ma1 -0.0032064  0.0439940 -0.0729   0.9419
## ma2  0.0353108  0.0455712  0.7748   0.4384
## ma3 -0.0713490  0.0481630 -1.4814   0.1385
```

```
fit_arima313 <- Arima(train_data, order = c(3, 1, 3))
cat("      HASIL COEFTEST ARIMA(3,1,3)\n")
```

```
##       HASIL COEFTEST ARIMA(3,1,3)
```

```
coeftest(fit_arima313)
```

```
##
## z test of coefficients:
##
##      Estimate Std. Error z value Pr(>|z|)
## ar1 -0.108112   0.441827 -0.2447 0.806694
## ar2 -0.612420   0.246770 -2.4817 0.013074 *
## ar3 -0.635490   0.427982 -1.4849 0.137583
## ma1  0.085196   0.444919  0.1915 0.848145
## ma2  0.664078   0.247215  2.6862 0.007226 **
```

```
## ma3  0.620018   0.437942  1.4158 0.156847
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Conclusion Only ARIMA(2,1,2) that significant

8.5.1 Assumption Testing

There are 2 assumption testing in ARIMA model: 1. Residual are white noise 2. Residual are normally distributed

8.5.1.1 Residual White noise

H_0: Residuals are white noise H_1: Residuals are not white noise α = 5% = 0.05

```
e = resid(fit_arima212)
Box.test(e)

##
##  Box-Pierce test
##
## data:  e
## X-squared = 0.039575, df = 1, p-value = 0.8423
```

Since the p-value is 0.8423, we fail to reject H_0. Therefore, the residuals are white noise and satisfy the model assumptions.

8.5.1.2 Uji Normalitas

H_0: Residuals are normally distributed H_1: Residuals are not normally distributed α = 0.05

```
e = resid(fit_arima212)
lillie.test(e)

##
##  Lilliefors (Kolmogorov-Smirnov) normality test
##
## data:  e
## D = 0.051048, p-value = 0.002251
```

Since the p-value is 0.002251, H_0 is rejected. Therefore, the residuals are not normally distributed and do not satisfy the assumption.Based on the results of the Lilliefors test on the ARIMA model residuals, the obtained p-value is < 0.05, which means the null hypothesis is rejected and the model residuals are not normally distributed. Non-normality in residuals is a common phenomenon encountered in financial and daily commodity time series data (asymmetric fat-tails). This indicates the presence of non-constant variance (heteroscedasticity) or high volatility driven by market shocks (structural breaks). This finding simultaneously provides a strong empirical justification that standard linear models such as ARIMA have limitations in capturing dynamic variance effects, thereby

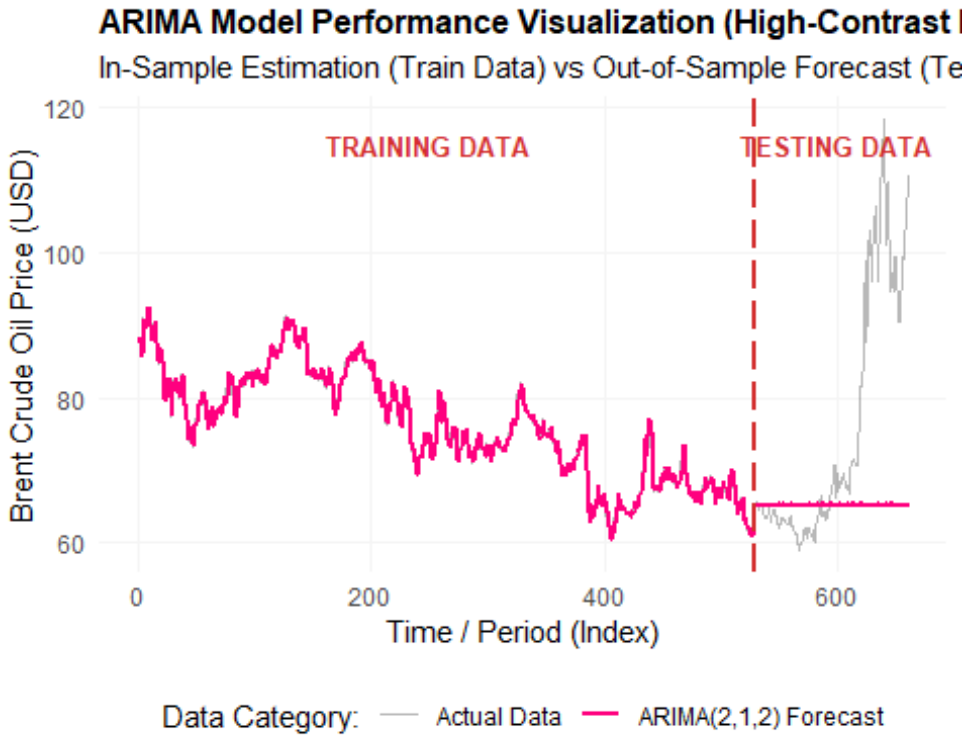
necessitating further analysis using volatility-based models such as ARCH/GARCH or non-linear Machine Learning-based approaches.

```
h_period <- length(ts_data) - train_size

forecast_arima212 <- forecast(fit_arima212, h = h_period)

akurasi_train_arima <- accuracy(fit_arima212$fitted, train_data)
akurasi_test_arima  <- accuracy(forecast_arima212$mean, test_data)

tabel_lengkap_arima <- rbind(
  "Training set" = akurasi_train_arima[1, ],
  "Test set"     = akurasi_test_arima[1, ]
)

cat("       TABEL AKURASI MODEL ARIMA(2,1,2)\n")

##        TABEL AKURASI MODEL ARIMA(2,1,2)

print(tabel_lengkap_arima)

##                      ME      RMSE       MAE         MPE      MAPE
ACF1
## Training set -0.0433558  1.348749  1.021795 -0.07314676  1.357465
0.008657485
## Test set     10.6504596 20.115698 12.885904 10.21912533 13.837705
0.963652254
##              Theil's U
## Training set 0.9940897
## Test set     5.6652759
```

8.5.2 Forecasting Plot

```
df_plot_arima <- data.frame(
  Indeks = 1:length(ts_data),
  Aktual = as.numeric(ts_data),
  Prediksi = c(as.numeric(fit_arima212$fitted),
as.numeric(forecast_arima212$mean))
)

ggplot(df_plot_arima, aes(x = Indeks)) +
  geom_line(aes(y = Aktual, color = "Actual Data"), linewidth = 0.6, alpha =
0.4) +
  geom_line(aes(y = Prediksi, color = "ARIMA(2,1,2) Forecast"), linewidth =
1.0, lty = 1, alpha = 1) +
  geom_vline(xintercept = train_size, linetype = "longdash", color =
"#d32f2f", linewidth = 0.8) +
  annotate("text", x = 250, y = 115, label = "TRAINING DATA", color =
"#d32f2f", fontface = "bold", size = 3.5) +
  annotate("text", x = 600, y = 115, label = "TESTING DATA", color =
"#d32f2f", fontface = "bold", size = 3.5) +
```

```
  scale_color_manual(values = c("Actual Data" = "gray30", "ARIMA(2,1,2)
Forecast" = "#ff007f")) +
  labs(
    title = "ARIMA Model Performance Visualization (High-Contrast Edition)",
    subtitle = "In-Sample Estimation (Train Data) vs Out-of-Sample Forecast
(Test Data) using ARIMA(2,1,2)",
    x = "Time / Period (Index)",
    y = "Brent Crude Oil Price (USD)",
    color = "Data Category:"
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    panel.grid.major = element_line(color = "#f5f5f5"),
    panel.grid.minor = element_blank(),
    plot.title = element_text(face = "bold", size = 12)
  )
```



9. Bandingkan Model

9.1 Tabel perbandingan

```
# --- MODEL 1: NAIVE ---
acc_naive_train <- accuracy(model_naive)[1, ]
acc_naive_test  <- accuracy(model_naive$mean, test_data)[1, ]

# --- MODEL 2: DMA(6) ---
```

```r
acc_dma_train <- accuracy(train_fitted_dma3,
                    train_data[(length(train_data)-
                        length(train_fitted_dma3)+1):length(train_data)])[1,]

acc_dma_test <- accuracy(test_forecast_dma3,test_data)[1, ]

# --- MODEL 3: DES(alpha = 0.16 dan beta = 0.081) ---
acc_des_train <- accuracy(model_des5)[1, ]
acc_des_test <- accuracy(model_des5$mean,test_data)[1, ]

# --- MODEL 4: ARIMA(2,1,2) ---
acc_arima_train <- accuracy(fit_arima212$fitted, train_data)[1, ]
acc_arima_test  <- accuracy(forecast_arima212$mean, test_data)[1, ]

# --- MODEL 5: TS REGRESSION ---
acc_reg_train <- accuracy(train_fitted_reg5, df_train_reg$Aktual)[1, ]
acc_reg_test <- accuracy(test_forecast_reg5, test_data)[1, ]

tabel_komparatif <- data.frame(
  Model = c("Naive", "Moving Average (DMA)", "Exponential Smoothing (DES)",
"ARIMA(2,1,2)", "TS Regression"),

  # Kolom RMSE
  RMSE_Train = c(acc_naive_train["RMSE"], acc_dma_train["RMSE"],
acc_des_train["RMSE"], acc_arima_train["RMSE"], acc_reg_train["RMSE"]),
  RMSE_Test  = c(acc_naive_test["RMSE"],  acc_dma_test["RMSE"],
acc_des_test["RMSE"],  acc_arima_test["RMSE"],  acc_reg_test["RMSE"]),

  # Kolom MSE
  MSE_Train  = c(acc_naive_train["RMSE"]^2, acc_dma_train["RMSE"]^2,
acc_des_train["RMSE"]^2, acc_arima_train["RMSE"]^2, acc_reg_train["RMSE"]^2),
  MSE_Test   = c(acc_naive_test["RMSE"]^2,  acc_dma_test["RMSE"]^2,
acc_des_test["RMSE"]^2,  acc_arima_test["RMSE"]^2,  acc_reg_test["RMSE"]^2),

  # Kolom MAPE
  MAPE_Train = c(acc_naive_train["MAPE"], acc_dma_train["MAPE"],
acc_des_train["MAPE"], acc_arima_train["MAPE"], acc_reg_train["MAPE"]),
  MAPE_Test  = c(acc_naive_test["MAPE"],  acc_dma_test["MAPE"],
acc_des_test["MAPE"],  acc_arima_test["MAPE"],  acc_reg_test["MAPE"])
)

cat("          COMPARATIVE TABLE ACCURACY MULTI METRIC (TRAIN VS TEST
SET)\n")
```

```
##           COMPARATIVE TABLE ACCURACY MULTI METRIC (TRAIN VS TEST SET)
```

```r
print(round(tabel_komparatif %>% tibble::column_to_rownames("Model"), 4))
```

```
##                               RMSE_Train RMSE_Test MSE_Train MSE_Test
MAPE_Train
## Naive                             1.3589   20.0724    1.8467 402.8996
1.3648
## Moving Average (DMA)              1.4807   20.6840    2.1924 427.8263
1.5229
## Exponential Smoothing (DES)       2.7065   10.4854    7.3254 109.9441
2.8004
## ARIMA(2,1,2)                      1.3487   20.1157    1.8191 404.6413
1.3575
## TS Regression                     3.5411   17.3762   12.5395 301.9329
3.7273
##                               MAPE_Test
## Naive                           13.8413
## Moving Average (DMA)            13.9676
## Exponential Smoothing (DES)     11.8643
## ARIMA(2,1,2)                    13.8377
## TS Regression                   12.5389
```

Based on the multi-metric comparison table on the test data set, the ARIMA(2,1,2) model produced the best performance in the MAPE_Test metric at 13.8377%, while the Naive model had a slight advantage in the RMSE_Test metric at 20.0724.

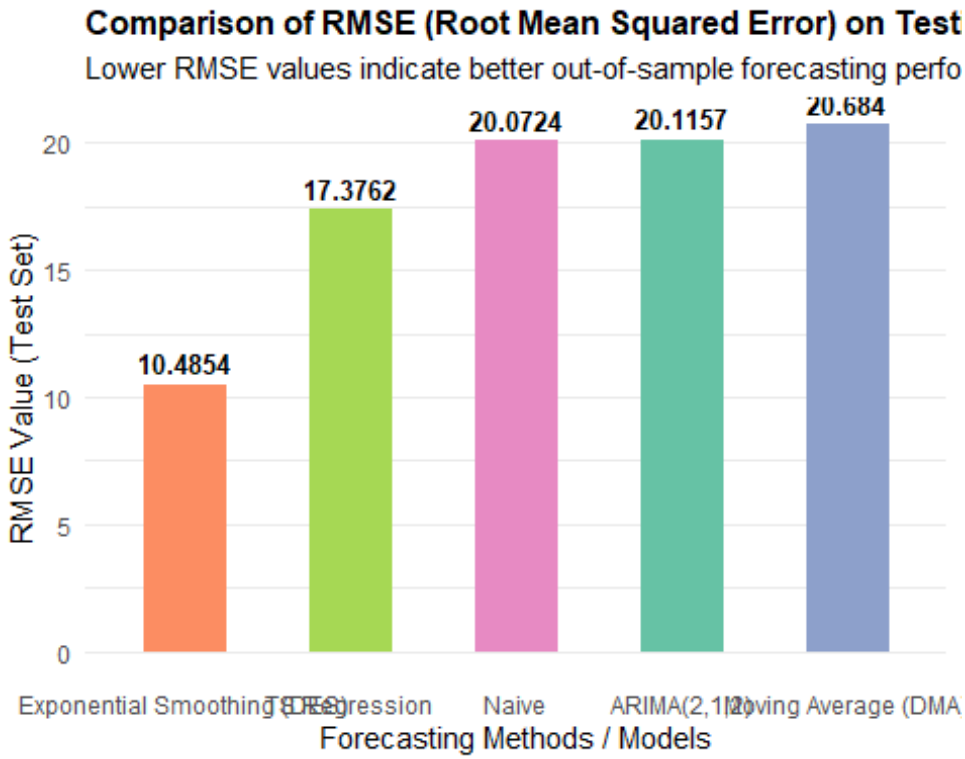
The significant spike in error values (RMSE and MAPE) across all models when switching from training to testing data was due to a structural break in the Brent oil market. At the end of the training period, the price closed at USD 65.29, but during the testing period, the market experienced an extreme bullish trend, peaking at USD 118.35.

Given that the ARIMA(2,1,2) model was able to minimize the overall average percentage deviation (the smallest MAPE), the ARIMA(2,1,2) and Naive models were selected as the best models for 30-day forecasting.

9.2 Visualisasi RMSE

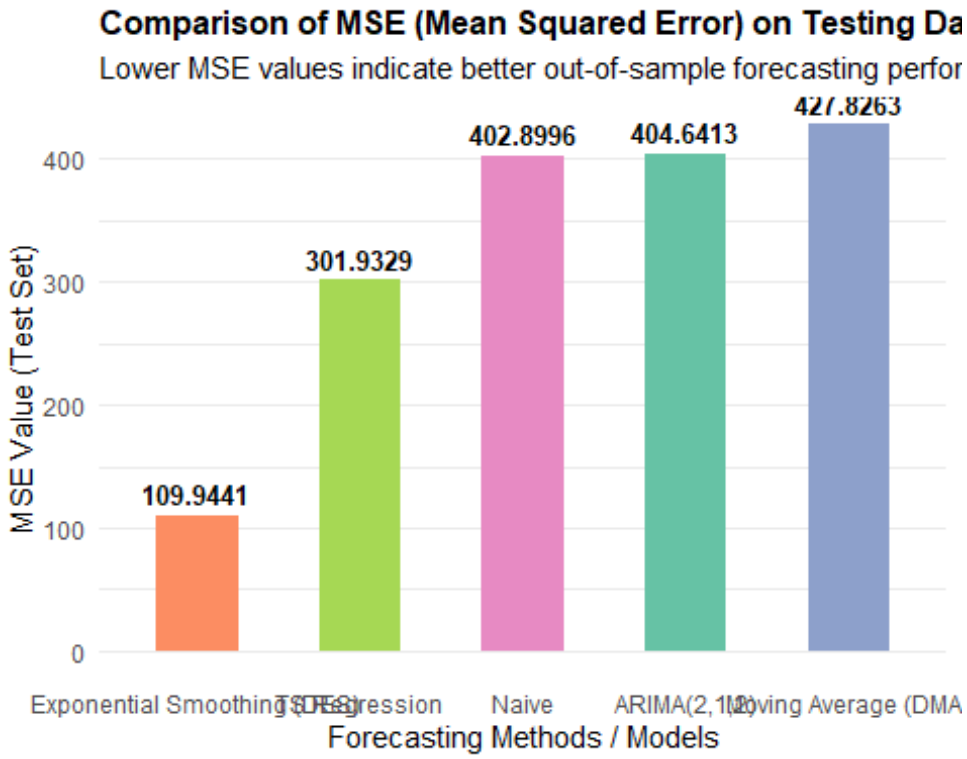
```
ggplot(tabel_komparatif, aes(x = reorder(Model, RMSE_Test), y = RMSE_Test,
fill = Model)) +
  geom_bar(stat = "identity", width = 0.5, show.legend = FALSE) +
  geom_text(aes(label = round(RMSE_Test, 4)), vjust = -0.5, fontface =
"bold", size = 3.5) +
  scale_fill_brewer(palette = "Set2") +
  labs(
    title = "Comparison of RMSE (Root Mean Squared Error) on Testing Data",
    subtitle = "Lower RMSE values indicate better out-of-sample forecasting
performance",
    x = "Forecasting Methods / Models",
    y = "RMSE Value (Test Set)"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 12),
```

```
    panel.grid.major.x = element_blank()
  )
```



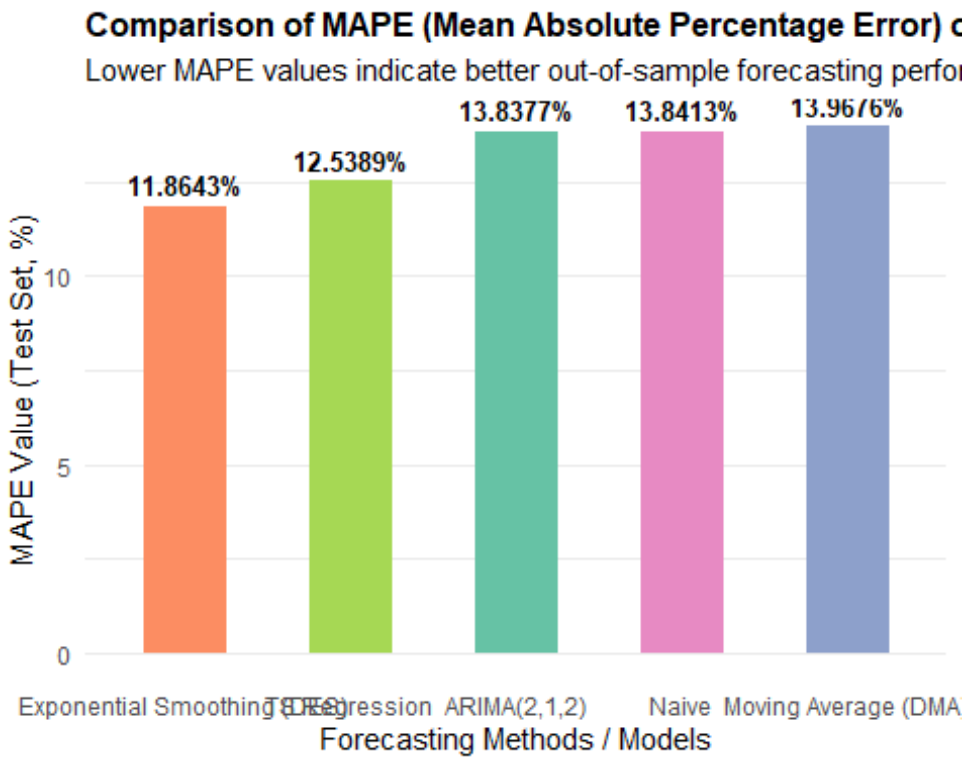
9.3 Visualisasi MSE

```
ggplot(tabel_komparatif, aes(x = reorder(Model, MSE_Test), y = MSE_Test, fill
= Model)) +
  geom_bar(stat = "identity", width = 0.5, show.legend = FALSE) +
  geom_text(aes(label = round(MSE_Test, 4)), vjust = -0.5, fontface = "bold",
size = 3.5) +
  scale_fill_brewer(palette = "Set2") +
  labs(
    title = "Comparison of MSE (Mean Squared Error) on Testing Data",
    subtitle = "Lower MSE values indicate better out-of-sample forecasting
performance",
    x = "Forecasting Methods / Models",
    y = "MSE Value (Test Set)"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 12),
    panel.grid.major.x = element_blank()
  )
```



9.4 Visualisasi MAPE

```
ggplot(tabel_komparatif, aes(x = reorder(Model, MAPE_Test), y = MAPE_Test,
fill = Model)) +
  geom_bar(stat = "identity", width = 0.5, show.legend = FALSE) +
  geom_text(aes(label = paste0(round(MAPE_Test, 4), "%")), vjust = -0.5,
fontface = "bold", size = 3.5) +
  scale_fill_brewer(palette = "Set2") +
  labs(
    title = "Comparison of MAPE (Mean Absolute Percentage Error) on Testing
Data",
    subtitle = "Lower MAPE values indicate better out-of-sample forecasting
performance",
    x = "Forecasting Methods / Models",
    y = "MAPE Value (Test Set, %)"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 12),
    panel.grid.major.x = element_blank()
  )
```


Comparison of MAPE (Mean Absolute Percentage Error) c
Lower MAPE values indicate better out-of-sample forecasting perfo
11.8643%
12.5389%
13.8377%
13.8413%
13.9676%
MAPE Value (Test Set, %)
10
5
0
Exponential Smoothing (SES)
ARIMA(2,1,2)
Naive
Moving Average (DMA
Forecasting Methods / Models